NØNOS

A computer that can prove what it runs

Release 0.9.3: proof for the software we ship, and a box for everyone else's

whitepaper on release 0.9.3 repository commit 832ab24778 30 October 2026

https://nonos.software https://github.com/NON-OS/microkernel-preview https://verify.nonos.software

Abstract

NØNOS is an operating system for x86_64 computers that tries to make its software state checkable, by the machine and by someone else, before that software is trusted with keys, money or traffic. A Rust capability microkernel keeps page tables, scheduling, IPC and 183 checked system calls in ring 0. Drivers, the network stack, the desktop and a wallet run in ring 3 as 152 signed capsules, each admitted at spawn by two signatures and a STARK membership proof under one policy root.

Release 0.9.3 adds what 0.9.2 left open: programs the project did not write. A Linux program opens its own window on the desktop, inside a box named by its signed Marketplace listing: a window, its own home folder, the chosen anonymity network, a terminal, the right to start programs. The program holds no capability of its own. The kernel sends every call it makes to one supervisor capsule, which applies the box, and keeps its own count of the answers. We show what the system refused when single hostile attempts were made from inside a program, and what it did not, and list the three known ways around the box.

The paper says what has been proved, what has been booted, and what is only designed. No independent audit has been done.

> **Reading this paper.** Every technical statement cites a file in the repository at the commit above, with line numbers where they help, in a numbered footnote. A claim we could not tie to the tree is not made. Where two documents in the tree disagree, we say so and follow the code. Section 1 states the problem. Sections 4 to 6 describe the kernel and how it admits software. Section 7 is the centre of this release: running programs we did not write. Section 12 says what has run and what has not. Nothing in this paper is an audit, and none has been done.

1 Why NØNOS exists

A computer cannot tell anyone, in a form that can be checked, what software it is running. A person who signs a transaction on a laptop trusts that the signing program is the one they meant to run, that no other program can read its key, and that the network path does not give them away. None of this is visible from outside, and little of it is visible from inside.

Public blockchains moved most of the trust in a payment system into code that anyone can check. Ethereum verifies every state transition with rules that every node enforces. The machines at its edges sit outside that system. A wallet key lives on a general-purpose computer with thousands of programs that can read its memory or its disk. A validator signs with keys on a server whose kernel and drivers nobody outside can see. A remote procedure call provider learns the address of every machine that asks it about an account. A mixnet hides traffic patterns only for the traffic that is actually sent through it, and an operating system that lets any program open a direct socket undoes the protection without telling the person.

NØNOS starts from the endpoint. Release 0.9.2 asked three questions of every piece of code before it runs. Which image is this, and is it one of the images a policy committed to? What may it reach once it runs? Which network carries what it sends? The answers are enforced in a small kernel and, where we could, made checkable by someone who does not trust the machine.

1.1 The problem 0.9.2 left open

Those three questions have a quiet assumption in them: that the code is ours. Every one of the 152 capsules in the 0.9.3 tree is built by the project, signed with two algorithms and enrolled under one policy root, so the proof at admission says something exact about it.[1] A person does not live in a machine like that. They want a password manager somebody else wrote, a chat client, an editor they have used for ten years, a Monero wallet, a PDF viewer. If the system will run only what the project signed, it is a demonstration. If it will run anything, then the proofs describe a small part of what is on the machine, and the rest is as trustworthy as a stranger's download.

Most systems settle this by trusting the program. They check that it came from a store, then give it the person's files, the network and the screen, because it asked for them. A permission dialog is the usual refinement, and it moves the decision to the one party least able to make it. A person cannot read a Qt application's system calls. They can read a short list of what it asks for, if the list is enforced.

1.2 The thesis

We make two kinds of statement, and keep them apart.

For software the project ships, NØNOS makes a statement about **identity**: this exact file, with exactly these rights, is a member of the set one policy root committed to. That is a proof, checked by the kernel at every spawn, and checkable by anyone from outside.

For software the project did not write, an identity proof would say much less, and would be easy to mistake for more. We cannot prove that a Qt application behaves. We can make a statement about **reach**: this program gets a window, a home folder and the Anyone network, and nothing else, and the part of the system that enforces that is small, is itself admitted by proof, and sees every call the program makes. The list is short, it is part of the program's signed Marketplace listing, and a person can read it before they install.

The argument of this paper is that a capability microkernel is the right place to make the second statement. A microkernel has already moved drivers, the network stack and the desktop out of ring 0, so nearly everything a foreign program could want is a message to a small, named capsule with a capability word. The kernel does not need to understand Linux to hold a Linux program to a box. It needs to give the program nothing of its own, send every call to one checker, and keep its own count of what happened. Section 7 works through that.

[1] `README.md`, section "Capsules".

The release has a second half to its thesis, which is about honesty. A box is a claim, and claims fail. So the paper reports what the kernel refused when we ran real programs against it, what it did not refuse because the program was granted the right, and which of the ways around the box we already know about.

1.3 What this paper covers

Sections 2 and 3 state the goals and the threat model. Sections 4 to 6 describe the kernel, the chain of trust and the capsule format, which 0.9.2 introduced and 0.9.3 extends: the tree grew from 91 capsules to 152, and from 18 driver capsules to 67. Section 7 is the new centre. Sections 8 to 10 cover cryptography, networking and verification. Section 11 keeps the 0.9.2 discussion of Ethereum as a direction, with what changed. Sections 12 to 15 give the status of each component, use cases, limitations and the work we have not done. Section 16 places the design among its neighbours.

2 Design goals

The project writes its rules in `docs/overview/design-principles.md` and `docs/overview/mission.md`. We restate them as eight goals, say where each is enforced, and say which of them 0.9.3 changes. A goal is only useful if a reader can find the code that holds it, so each one points at a file.

G1. Least privilege by capability word. Every process holds a mask over 40 capability bits, fixed from its signed manifest at spawn, and every system call is checked against it. In 0.9.2 the mask had 36 bits. The four added since are `InputConsumer`, `CameraCapture`, `MicrophoneCapture` and `ScreenCapture`.[2]

G2. Everything that runs is signed and measured. The loader checks the kernel. The kernel checks the loader and every capsule. Admission needs two signature algorithms and a STARK membership proof.[3]

G3. The verifier is on the machine, the prover is elsewhere. The kernel and the loader link only `nox_verify`. Proofs are made on the enrolling host by `nonos-stark-enroll`.[4]

G4. Drivers run in ring 3. A driver is a capsule that reaches hardware only through grants from the kernel's broker, with DMA confined by the IOMMU where a remapping unit is in service. There are 67 driver capsules in 0.9.3.[5]

G5. Amnesic by default. A boot keeps nothing unless the person installs, and an orderly power-off wipes memory first. In 0.9.3 the desktop's Power Off item and the Power key both run that wipe.[6]

G6. Refuse rather than fall back. A chosen anonymity network that is not running means no route. Nothing is retried in the clear.[7]

G7. Proofs sit next to the code. Host proof crates compile the shipped source through `#[path]`, and Lean 4 models state the properties the code owes.[8]

G8 (new in 0.9.3). A foreign program holds nothing of its own. A Linux program is a process with an empty capability word. Everything it can do, it does through a supervisor that holds a box, and the box is part of the program's signed listing.[9]

[2] `src/capabilities/types/defs.rs:17-76`; `src/syscall/contract/dispatch.rs:31-45`

[3] `docs/security/boot-chain-and-signatures.md`

[4] `Cargo.toml:1444`; `nonos-stark-enroll/Cargo.toml:15-19`

[5] `docs/overview/architecture-hardware-broker-iommu.md`

[6] `nonos.toml:25-28`; `src/security/zerostate/terminate.rs:23-45`

[7] `userland/nonos_route_link/src/pick.rs:130-139`

[8] `verification/lean/REFINEMENT.md`

[9] `src/process/foreign/spawn_guest.rs:41-48`; `userland/nonos_linux_policy/README.md:1-12`

The project also says what it does not claim: functional correctness of the whole kernel, hardware breadth, anonymity against every observer, and an independent audit.[10] We add one. G8 is a goal, not a result. Section 7 says how far the evidence goes.

3 System and threat model

3.1 System model

The machine is an x86_64 computer with UEFI firmware, optionally a TPM 2.0, and optionally an Intel VT-d or AMD-Vi remapping unit. The loader `BOOTX64.EFI` starts the kernel. The kernel starts its capsules from the kernel image and from a signed store. A build is one of seven profiles: standard, hardened, airgapped, qemu, dev, selftest and core. A release image is built hardened, which requires Secure Boot and a TPM at boot.[11]

3.2 Assets

The threat model lists what it protects and what guards each asset.[12] Five matter here. The wallet's key, held by the keyring capsule. The record of what runs, held by signatures, attestation and the capability word. The device secret, derived from the TPM and released to one capsule. The data volume, sealed under a key the TPM derives from the current boot state. And, new in 0.9.3, the person's own files and the contents of other programs' memory, which a Linux program must not reach unless its listing says it may.

3.3 Adversaries in scope

- A capsule, an app or a Linux program that tries to reach past its capability word or its box.
- A device that attempts DMA outside its capsule's grants, where a remapping unit in service covers it.
- An observer on the network path, for connections made under Nym or Anyone.
- Someone who reads the disk of a machine that is off.
- A capsule or kernel changed after it was signed and enrolled; a loader changed after enrolment that still hands the kernel the TPM event log; an older signed kernel put back on a machine whose TPM holds the rollback floor.

This is the project's own list.[13] The first item is the one 0.9.3 widens. The Linux programs we have in mind are not hostile by default. A KeePassXC build from a distribution is probably fine. But the point of a box is that nobody has to decide that, and the adversary we design against is a program that behaves as a hostile one would: reads the person's files, signals other programs, opens a socket to somewhere it was not told about, starts a tool it was not given. We also design against the plainer case, a bug in a program that a hostile document can steer.

What a hostile Linux program gets is spelled out in the threat model. It holds no capabilities. The kernel parks each of its Linux system calls for the personality to answer, and refuses every native call except `MkTtyQuery`, which only describes the caller's own streams. The personality's ordinary run and terminal instances hold no Network capability, and every network service requires it.[14]

The threat model also says the personality is itself a capsule, so a program that subverts it gets the personality's bits and no more. Those are CoreExec, IPC, Memory, Crypto, FileSystem, Debug, two

[10] `docs/overview/mission.md`, section "What NONOS is not".

[11] `README.md`, section "Build it"; `docs/build/profiles.md`.

[12] `docs/overview/threat-model-assets.md`

[13] `docs/overview/threat-model-in-scope.md`

[14] `docs/overview/threat-model-assets.md`, section "A hostile Linux program"; `src/process/foreign/trap.rs:28-57`; `src/syscall/contract/cap_table/mk_surface.rs:60`.

display bits, StoreWrite, ForeignExec and LocalSign, with Network added in the install role.[15] That is a long list, and it is the honest answer to "how strong is the box": as strong as the supervisor that enforces it, plus the kernel's refusal to let a guest do anything on its own. Section 7.11 comes back to this.

3.4 Assumptions and exclusions

NØNOS trusts the CPU, the UEFI firmware until `ExitBootServices`, and the TPM. It assumes the IOMMU translates as its tables say, RDRAND and RDSEED are unpredictable, BLAKE3, SHA-2, SHA-3 and the width-8 Poseidon hash are collision resistant, and the Rust compiler is correct.[16] Out of scope are lying firmware or TPMs; physical access to a running machine; DMA from a device that no remapping unit covers; timing and cache side channels between capsules; a kernel panic, which wipes the kernel heap and powers off but not process memory; and a power cut, which wipes nothing.[17]

Without a TPM, the rollback floor is absent in every boot mode except Hardened and Air-Gapped, and the kernel's check of its loader rests on the loader's own word.[18]

Compared with 0.9.2, two exclusions moved. AMD-Vi is now driven in every build profile, so "every device on AMD-Vi platforms" is no longer outside the model, although it has run only under QEMU's AMD IOMMU and not on an AMD machine. And on a machine whose firmware left the remapping unit off, a Standard boot now says so on its `[IOMMU]` line, in the attestation, on the splash and in About, and Hardened and Air-Gapped stop instead.[19]

4 Architecture

4.1 What stays in ring 0

The kernel is written in Rust. At the commit studied, `src/` holds 7,579 Rust files and 389,695 lines (our count, with `git ls-tree` and `wc`; the README says about 390,000). In 0.9.2 the same count gave 5,139 files. We have not measured where the growth sits. What we can say is what ring 0 still keeps: only address spaces and page tables, the scheduler with symmetric multiprocessing up to 256 CPUs, IPC, capabilities, the system-call boundary, process creation and the capsule spawn gate. It also keeps the TPM driver and the encrypted data volume, a hardware broker that hands device windows to driver capsules, and the kernel's record of what each Linux guest did (Section 7).[20]

Everything else runs in ring 3. The tree has 152 directories with a `Capsule.mk`, up from 91 at the 0.9.2 changelog, and 67 of them are driver capsules, up from 18 (Table 1). Figure 1 shows the split.

The first four rows are the owner's counts and we reproduced them from the git trees. The last two come from the ABI files, which a checker holds to the kernel.[21] Run on this tree, the checker reports "183 published syscalls reach a handler" and `check_caps_abi.py` reports "40 published bits agree with the kernel". A word on the 0.9.2 figures: that paper counted at a later commit than the changelog, and its own count of kernel files was 5,758. We use the changelog commit for both columns so that the comparison is of like with like.

[15] `userland/capsule_linux/Capsule.mk:50-59`; `src/userspace/capsule_linux/spawn.rs:39-55`

[16] `docs/overview/threat-model-a-tampered-image.md`, section "Assumptions".

[17] `docs/overview/threat-model-in-scope.md`; `docs/release/0.9.3-security-limits.md`

[18] `docs/security/protections-and-limits-what-nonos-does.md`

[19] `docs/release/0.9.3-supported-hardware.md`; `docs/release/0.9.3-security-limits.md`

[20] `src/smp/constants.rs:17`; `docs/overview/architecture-kernel-in-ring.md`

[21] `abi/syscalls.toml`; `scripts/check_syscall_abi.py`

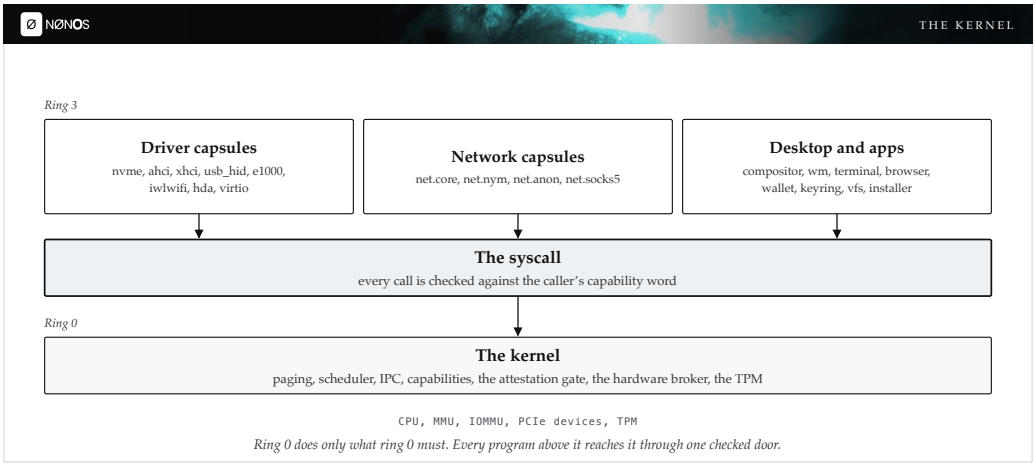


Figure 1. The kernel and everything above it. Ring 3 holds drivers, network services, the desktop and every program; ring 0 holds what only ring 0 can do. Every program reaches the kernel through one checked door, the system call. Drawn from `docs/overview/architecture.md` and the capsule list in the tree.

Table 1. What grew between the 0.9.2 changelog and the 0.9.3 tree. Counted from git at the commit where `CHANGELOG-0.9.2.md` landed (`76a809a8a2`, 6 October 2026) and at the studied commit. About 6,000 commits separate them. Capsules are directories with a `Capsule.mk`; proof crates are directories ending in `_proofs` with a `Cargo.toml`.

	0.9.2	0.9.3
Driver capsules	18	67
Capsules in all	91	152
Rust files in the kernel, `src/`	5,139	7,579
Rust files in `userland/`	10,069	31,674
Proof crates	56	223
System calls	130	183
Capability bits	36	40

4.2 The system-call ABI

A call number is four ASCII letters packed into a 64-bit word. `MkIpcSend`, for example, is `MISD`. Arguments travel in `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9`. A result between -4095 and -1 is a negative errno.[22] The kernel defines 183 calls: 167 microkernel calls, 12 cryptographic, 3 administrative and 1 for graphics. The count is the same three ways: 183 tags under `[numbers]` in `abi/syscalls.toml`, 183 variants of `SyscallNumber`, and a registry the checker confirms reaches a handler on every one.[23] The documentation page for system calls still says 176 and 147 in places; it was last counted before the latest additions, and we follow the code.[24]

A number the kernel does not know is the case that matters most for this release. On x86_64 it is answered with `ENOSYS`, unless the caller is a Linux guest, in which case the kernel parks the guest and wakes its supervisor. A Linux system call number never collides with a NØNOS one, because NØNOS numbers are four letters and Linux numbers are small integers.[25]

[22] `abi/syscalls.toml:9-19`; `src/syscall/abi/tag.rs:17-22`

[23] `abi/syscalls.toml:55-242`; `src/syscall/numbers/defs.rs:6-191`

[24] `docs/kernel/syscalls.md`

[25] `src/process/foreign/trap.rs:28-57`; `docs/kernel/syscalls.md`

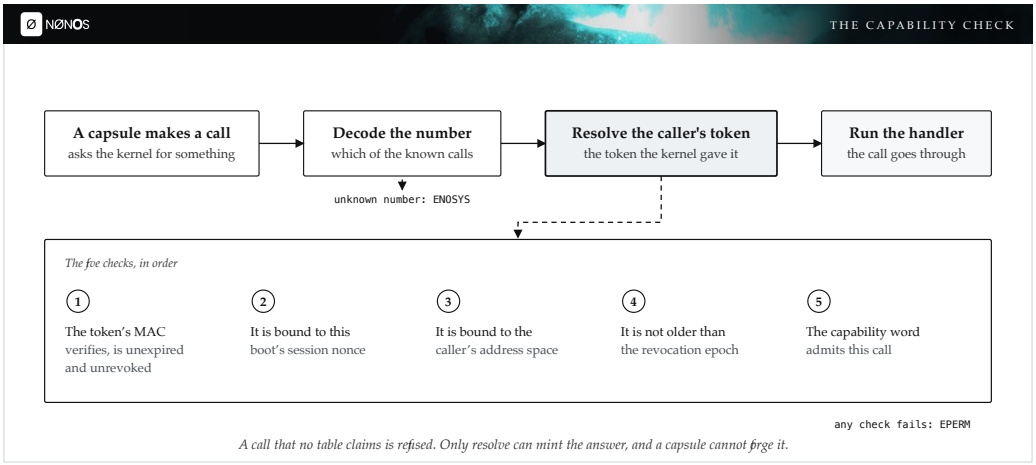


Figure 2. The capability check on every native system call: decode the number, resolve the caller's token, run five checks in order. Any failure is `EPERM`. A number that is not a native call is `ENOSYS`, or goes to the guest's supervisor (Section 7). Sources: `docs/kernel/capabilities-the-check-on.md` and `src/syscall/contract/resolver/resolve.rs`.

4.3 The capability word and the check path

Capabilities are 40 bits, defined once in `src/capabilities/types/defs.rs` and published in `abi/caps.toml`, which `scripts/check_caps_abi.py` holds to the kernel list. Bits range from `CoreExec` (bit 0) through `Network`, `IPC`, `Memory`, the device bits, to `DeviceSecret` (bit 35) and the four newer ones. Each process holds a token bound to its pid, its address-space id, the boot-session nonce and a revocation epoch, authenticated by a keyed BLAKE3 MAC whose key only the kernel holds.[26]

Every architecture's entry hands each known call to one dispatch function, which calls `Capability::resolve` and returns `EPERM` on failure. `resolve` runs five checks in order: the token's MAC, expiry and revocation; the boot-session binding; the address-space binding; the revocation epoch; and whether the cap table admits this call for this word.[27] The cap table is total: a call that no table claims is refused. Figure 2 draws the path.

At spawn the kernel installs the word from the verified manifest exactly once. A second call is rejected, so a stale spawn path cannot reissue authority. A request for a bit the manifest does not declare is refused with `GrantOutsideManifest`.[28] A new process inherits only a small ambient set from its parent, and an air-gapped boot removes `Network` from every capsule.[29]

The documentation is candid about what the word does not do yet. The `IO` and `Hardware` bits gate nothing. `Keyring` and `Entropy` gate only kernel clients that no code calls. And the image-wide capability ceiling is computed and logged, not enforced: a capsule that asks for more still starts, and the log prints `[CEILING] not enforced, would refuse`.[30]

4.4 IPC

Each capsule has one inbox, and a service is an endpoint in a registry with the capability bits a sender must hold. A send by name passes four gates: the `IPC` bit in the token; the endpoint's required bits; the held-endpoint and peer lists, which restrict who may reach a driver; and a byte budget.[31] The kernel writes the sender's name itself, so a capsule cannot forge it. A payload is at most 1 MiB; all inboxes

[26] `src/process/caps.rs:38-58`; `src/capabilities/token/material.rs:41-52`

[27] `src/syscall/contract/dispatch.rs:31-45`; `src/syscall/contract/resolver/resolve.rs:31-43`

[28] `src/process/caps.rs:115-127`; `src/security/capsule_manifest/verify/caps.rs:31-39`

[29] `src/process/core/table/inherit.rs:52-61`; `src/kernel_core/process_spawn/capsule_spawn/runner/profile_gate.rs:48-56`

[30] `docs/kernel/capabilities-what-each-bit.md`; `docs/build/profiles.md`

[31] `docs/kernel/ipc-who-may-send.md`

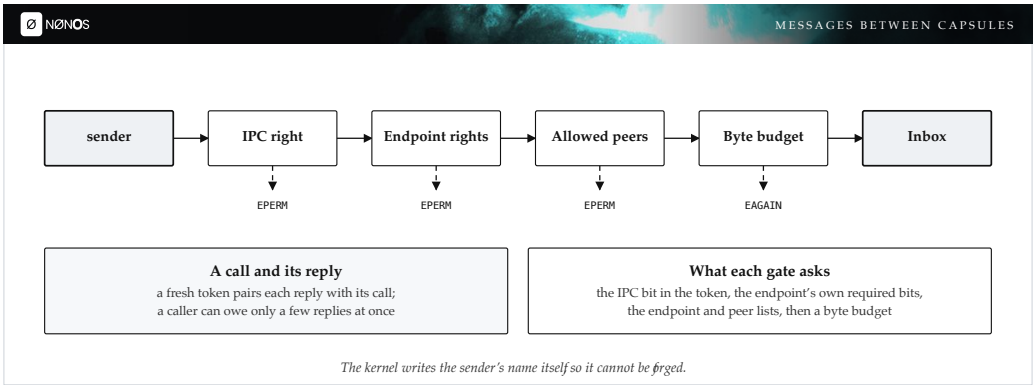


Figure 3. The four gates on a send between capsules, with the error each returns. The kernel stamps the sender's name. Sources: `docs/kernel/ipc-who-may-send.md`, `src/ipc/nonos_inbox/budget.rs`.

Table 2. The 67 driver capsules by class, generated from the `userland/capsule_driver_*` directories that hold a `Capsule.mk`. Which of them have run on real hardware and which only in their host model is in the hardware notes, and the answer for most is the second.

Class	Count	Drivers
Storage	12	`ahci`, `nvme`, `piix_ide`, `buslogic`, `mpt`, `pvscsi`, `storvsc`, `ufs`, `sdhci`, `rtsx`, `virtio_blk`, `virtio_scsi`
USB	17	`xhci`, `ehci`, `ohci`, `uhci`, `usb_hub`, `usb_hid`, `usb_msc`, `usb_audio`, `usb_mic`, `uvc`, `ccid`, `fido`, `cdc_acm`, `cdc_ecm`, `cdc_ncm`, `rndis`, `bt_usb`
Wired network	14	`e1000`, `e1000e`, `igb`, `igc`, `ixgbe`, `rtl8139`, `rtl8153`, `rtl8169`, `ax88179`, `pcnet`, `tulip`, `virtio_net`, `vmxnet3`, `netvsc`
Wi-Fi	5	`iwlwifi`, `mt7921`, `rtl8821ce`, `rtl8822ce`, `rtl8852be`
Input	5	`ps2_input`, `i2c_hid`, `spi_hid`, `virtio_input`, `vmmouse`
Display	5	`bga`, `bootfb`, `intel_display`, `virtio_gpu`, `vmware_svga`
Audio	2	`hda`, `ac97`
Buses	4	`gpio`, `i2c_pci`, `spi`, `smbus`
Virtual machine guests	3	`virtio_rng`, `vmbus`, `vmmdev`
Total	67	

together hold at most 96 MiB, one inbox 16 MiB, and one sender at most half an inbox.[32] Reaching any of the 11 services that carry traffic off the machine takes `Network` as well as `IPC`.[33] The peer list still has one entry: the Shield prover may send only to `shield.core`.[34] Figure 3 shows the gates.

4.5 Drivers and the hardware broker

A driver capsule claims its device from the broker and then asks for MMIO windows, DMA buffers, interrupt bindings and port I/O, each behind its own capability bit. The broker gives each driver capsule its own IOMMU domain where a unit is in service, and every image turns translation on at boot. Where no unit is in service, a Standard boot goes on with the claim unconfined and says so in four places; Hardened and Air-Gapped stop.[35] On release the broker stops bus mastering before it frees DMA frames.[36]

[32] `src/ipc/nonos_inbox/budget.rs:41-43`
[33] `src/services/registry/policy.rs:26-38`
[34] `src/services/registry/peers.rs:27-35`
[35] `docs/kernel/iommu-per-capsule-domains.md`; `docs/release/0.9.3-security-limits.md`
[36] `docs/overview/threat-model-a-hostile-or.md`

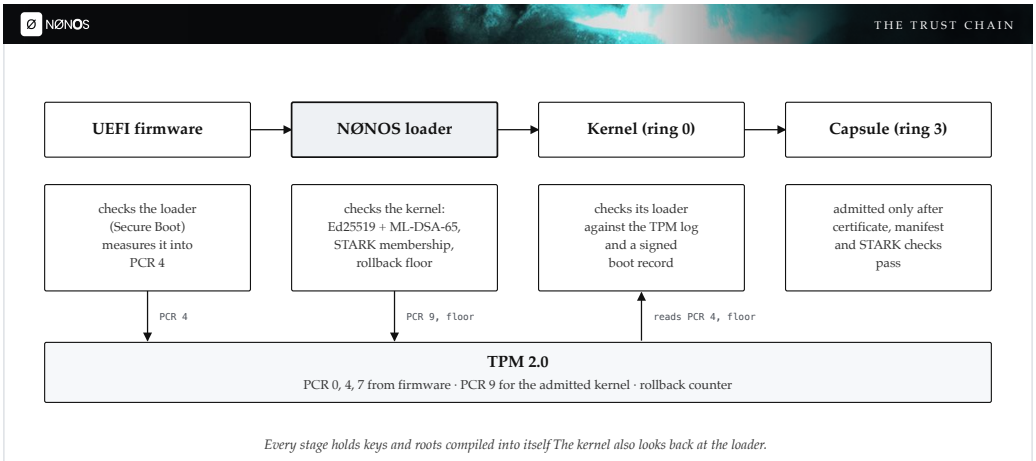


Figure 4. The boot chain and the TPM registers each stage uses. Firmware measures the loader into PCR 4; the loader checks the kernel and extends PCR 9; the kernel checks the loader back against the TPM's event log and a signed boot-root record, then checks every capsule. Sources: `docs/security/boot-chain-and-signatures.md`, `docs/security/measured-boot-and-tpm.md`.

[37]The 0.9.2 changelog tree had 18 driver capsules, so 49 arrived between the two trees. Each is a ring-3 process with a claim on its own device. A driver whose device is missing leaves at once, and one whose device fails gives up after a few seconds.[38] We say clearly what this does not mean. Most of the new drivers have been exercised against a model of the chip or under QEMU. On QEMU the e1000, e1000e, igb, RTL8139, PCnet, vmxnet3 and Tulip drivers each obtained a DHCP lease; the RTL8821CE has one hardware report; the other Wi-Fi drivers have run only against modelled devices.[39]

5 The trust chain

Each stage holds the next to keys and roots compiled into itself, and the kernel also looks back at the loader (Figure 4). This part of the design is unchanged from 0.9.2 in structure. What 0.9.3 adds is boots, with a caution about which ones count. The development images that most of the evidence directories use carry path-only trailers, with no STARK proof, so they exercise the Merkle-path half of the gate and not the proof half. The proof half has run on sealed release images: the owner's sealed boot of 26 October has every capsule spawned and attested as `vendor`, and the screenshot below is a sealed image under KVM. The 0.9.2 paper could say only that the release was built and held by host tests.[40]

5.1 The loader checks the kernel

The kernel image carries a signature blob (`NKRSIG2`) with an Ed25519 signature and an ML-DSA-65 signature, each over the same 36-byte message: the BLAKE3 hash of the kernel ELF followed by the rollback index.[41] The public halves are compiled into the loader, and a production loader refuses to build without the ML-DSA-65 key.[42] The loader's pipeline runs the signature and rollback check, then `attest_kernel` for the STARK trailer, then raises the TPM rollback floor, and only then hands over.[43] A kernel whose trailer fails is refused in every mode, development included.[44]

[37] `docs/release/0.9.3-supported-hardware.md`; `docs/hardware/MATRIX.md`.
[38] `README.md`, section "Drivers".
[39] `docs/release/0.9.3-what-is-new.md`, section "Drivers"; `docs/release/0.9.3-known-issues.md`, section "Hardware".
[40] `docs/release-evidence/0.9.3/README.md`; `docs/release-evidence/0.9.3/owner-sealed-crash/README.md`
[41] `nonos-bootloader/tools/sign-kernel/src/message.rs:17-22`
[42] `nonos-bootloader/build.rs:135-216`
[43] `nonos-bootloader/src/entry/pipeline.rs:30-50`
[44] `nonos-bootloader/src/boot/attestation/kernel_gate.rs:34-60`



Figure 5. The loader's screen on the sealed 0.9.3 image, under KVM, with no TPM and Secure Boot off, both shown in orange at the top. Steps 5 to 7 measure the kernel with BLAKE3, check the Ed25519 and ML-DSA-65 signatures, and check the kernel's STARK proof. This is one boot of one image on a virtual machine; it shows the kernel proof checked and says nothing about the TPM path or about a machine with Secure Boot on.

Figure 5 is the loader's screen on the sealed 0.9.3 image. It lists the ten steps, and step 7 is "Kernel STARK proof". The screen also says what was not there: Secure Boot off, no TPM, "TPM OFF: no TPM 2.0 measured boot". That run was under KVM, which gives the guest no TPM, so the chain's TPM half did not run in it.

5.2 The kernel checks its loader

Before init, `check_bootloader` replays the firmware's TCG event log, requires it to reproduce the live PCR 4, takes the last application the log names as the loader, and holds that Authenticode digest to a 104-byte boot-root record signed with an ECDSA P-256 device-policy key compiled into the kernel, and to the loader's own STARK slot.[45] No program starts if the check refuses or finds no evidence.[46] Without a TPM or a log, the kernel falls back to hashing the loader file the loader handed it, and says so in its verdict. That check then rests on the loader's own word.[47]

5.3 The kernel checks every capsule

A capsule ships its ELF with a NØNOS ID certificate signed by the trust anchor, a manifest signed by its publisher, and an attestation trailer. `preflight` runs `verify_id_cert`, `verify_with_publisher` and `attest_gate` in that order. Certificates and manifests need one valid signature of each algorithm. The manifest check covers eight conditions, from the certificate hash to the declared endpoints.[48] Section 6 goes through admission.

5.4 The STARK gate and its statement

Every gate checks one statement: under root R, the leaf built from context C with kind K is a slot of a 256-slot tree. The gate builds C itself from what it is about to run and grant. For a capsule, C is 48 bytes: the BLAKE3 hash of the ELF, the capability word (8 bytes) and the policy epoch (8 bytes). For a kernel or loader, C is the measurement and the boot epoch.[49] The leaf is a Poseidon permutation

[45] `nonos-boot-measure/src/gate/verdict.rs:22-58`; `docs/security/boot-chain-and-signatures.md`, section "The kernel checks its loader".

[46] `src/kernel_core/init/entry/loader_refusal.rs:27-48`

[47] `nonos-boot-measure/src/gate/verdict.rs:60-75`; `src/security/boot/loader_check/run.rs:34-55`

[48] `src/kernel_core/process_spawn/capsule_spawn/runner/preflight.rs:36-76`; `src/security/capsule_manifest/verify/mod.rs:37-62`

[49] `nonos-attest-path/src/context.rs:23-40`

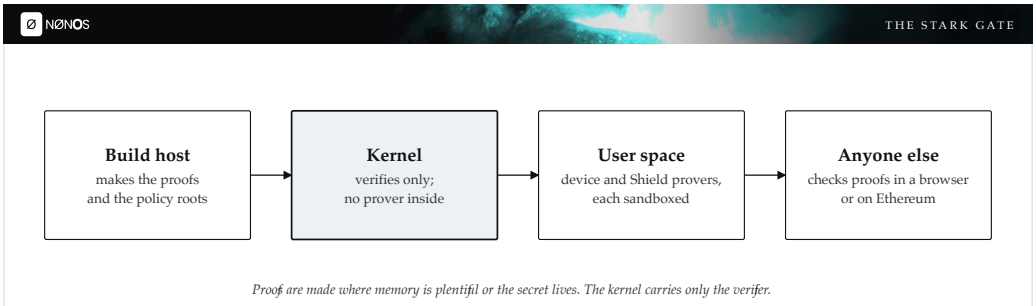


Figure 6. Who proves and who verifies. Proofs are made where memory is plentiful or where the secret lives. The kernel carries only the verifier. Sources: `README.md`, `docs/security/stark-attestation.md`.

over the Goldilocks field of a domain word, the digest of C and the kind, so a capsule slot cannot open as a kernel.[50]

The gate checks the statement twice, once by folding the Merkle path in the trailer up to the root, and once by running `nox_verify` on a transparent STARK proof of the same slot. It admits only when both pass, and the root is always the gate's own compiled-in or signed value, never the trailer's.[51] The proof system is hash-based, with a Fiat-Shamir transcript, no trusted setup and no pairing. A trailer carries at most 256 KiB of proof.[52]

Three trees exist: the capsule policy tree, checked by the kernel at every spawn; the kernel tree, checked by the loader; and the bootloader tree, whose root is signed into the boot-root record because it cannot live inside the loader it measures.[53] `nonos-stark-enroll` builds them on the enrolling host, checks every proof with the gates' own verifier before writing it, and records a transcript from which anyone can recompute the root.[54] Figure 6 shows where proofs are made and where they are checked.

A trailer does not prove that the image is signed, that the capsule's code behaves, which slot it occupies (the path travels in the clear), or freshness by epoch. The path check rests on Poseidon's collision resistance and the proof's soundness on the STARKs repository, and no proof in the tree covers either.[55] For this paper the middle item matters most. A STARK of membership is an identity statement. It says nothing about what a program does once it runs, and that is why Section 7 exists.

5.5 Measured boot, sealed keys and quotes

The TPM never stores a NØNOS key. The kernel asks it to derive keys under a PCR policy each time. The machine key, an HMAC under a primary key whose policy is the current value of PCRs 0, 4, 7 and 9, protects the data volume and sealed records, so the same disk in another machine, or the same machine in another boot state, gets another key.[56] The device secret is derived under a policy the release signs over PCR 9, combined with this machine's PCRs 0, 4 and 7, so an approved kernel update keeps it while a firmware or loader change does not.[57]

Two things are new in 0.9.3. A data volume formatted on a boot the release approved now has a random key, wrapped under a key the TPM gives only to a kernel the release approved. And an installed disk can be updated in place: from the moment the installed system prepares an update until the next boot, a one-boot handover record carries the key across a new bootloader. On QEMU with a software TPM, a prepared volume opened under an image with a changed bootloader through the

[50] `nonos-attest-path/src/leaf.rs:51-71`
[51] `src/security/capsule_attest/path.rs:45-53`
[52] `nonos-attest-path/src/v4/layout.rs:32`
[53] `docs/security/stark-attestation.md`, section "Three trees".
[54] `nonos-stark-enroll/src/stark/prove.rs:28-67`
[55] `docs/security/stark-attestation.md`, section "What a trailer does not prove".
[56] `src/security/tpm/machine_key/derive.rs:32-57`; `src/security/tpm/machine_key/pcrs.rs:21-24`
[57] `src/security/tpm/device_secret/derive.rs:33-54`

handover, and then under that bootloader's own key. The window in which the handover record exists is a known weakness, listed in Section 14.[58]

`MkAttestDoc` answers a 32-byte challenge with a TPM quote over PCRs 0, 1, 2 and 7. Because the attestation key is the same on every boot, a quote names the machine, so the kernel releases it only to a capsule that holds `AttestRead` and does not hold `Network`.[59] The quote covers firmware state and not the PCRs NØNOS itself extends; a design to quote the NØNOS chain exists and is marked as design, with nothing landing before the 0.9.3 seal.[60]

5.6 The anonymous device proof

`nonos-device-attest` proves, to a verifier who learns nothing else, four facts together: an approved bootloader started the machine, an approved kernel runs on it, the machine is an enrolled device, and a tag is this device's tag for the verifier's scope. The proof is made on the booted machine in a capsule that has no network bit and is the only one allowed `DeviceSecret`.[61] Its own README lists it as built and never run on a booted image, and we do not claim otherwise.[62]

6 Capsules

6.1 What a capsule is

A capsule is a statically linked position-independent ELF built offline for the target `x86_64-nonos-user`, plus three files: a NØNOS ID certificate, a binary manifest and an attestation trailer. Authors do not write manifests. They set `CAPSULE_*` variables in a `Capsule.mk`, and the build produces the manifest.[63] There are 152 `Capsule.mk` files in the tree. Each names the capsule's service, its endpoints and its capability word, and the same facts appear again in the kernel's own copy, which a script holds to the manifest.[64]

6.2 The manifest

The manifest is schema version 3 and big-endian. In wire order it carries the schema version; a certificate id, the BLAKE3 hash of the NØNOS ID certificate; a namespace of 1 to 96 bytes; a version; the target triple; a payload hash, the BLAKE3 hash of the whole ELF; the required capability word; the optional word, which must not overlap the required one; up to 16 endpoints; and one to four signatures.[65] The publisher signs every byte before the signature count. Algorithm 1 is Ed25519 with 64-byte signatures and algorithm 3 is ML-DSA-65 with 3309-byte signatures.[66]

6.3 Admission

Figure 7 shows admission at spawn. The certificate must verify under the trust anchor with both algorithms, must not be stale or revoked, and bounds what the publisher may request through a capability ceiling and namespace globs. The manifest must then verify under the publisher's keys with

[58] `docs/release/0.9.3-what-is-new.md`, section "Update NØNOS (keep my data)"; `docs/release-evidence/0.9.3/s1-k1-update/README.md`.

[59] `src/syscall/caps/checks/system.rs:40-52`

[60] `docs/design/k1-attest-doc-v4.md`

[61] `nonos-device-attest/README.md`; `userland/capsule_prove/Capsule.mk:17-35`

[62] `nonos-device-attest/README.md`, section "What is not done".

[63] `docs/userland/README.md`; `docs/userland/manifests-and-capabilities.md`

[64] `scripts/check_mirror_caps.py`

[65] `src/security/capsule_manifest/schema/constants.rs:17-25`; `src/security/capsule_manifest/decode/mod.rs:25-47`

[66] `src/crypto/asymmetric/alg_id/lengths.rs:19-26`

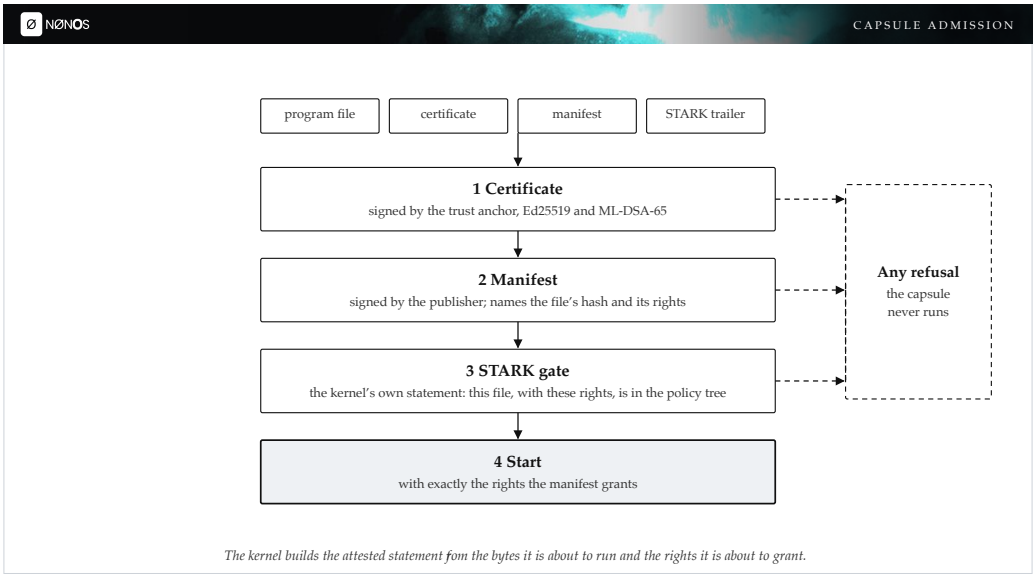


Figure 7. Capsule admission. The kernel builds the attested statement from the bytes it is about to run and the rights it is about to grant. Any refusal means the capsule never runs. Sources: `docs/security/boot-chain-and-signatures.md`, `docs/security/stark-attestation.md`.

both algorithms, and its payload hash must equal the BLAKE3 of the ELF being loaded. Finally the STARK gate checks the capsule's slot under the compiled-in policy root, with the manifest's required capability word as part of the statement. A capsule enrolled with one set of rights therefore cannot open with another.[67] Only after all three checks does the kernel create the process, install the word and record the measurement in the attestation registry.

A second admission path exists for code built on the machine itself. When the vendor root refuses a capsule, the kernel tries roots the person enrolled locally, and under such a root a trailer is a keyed tag the kernel minted for a local build instead of a STARK proof. The serial line `[ZK-ATTEST] ok` names the authority that admitted each capsule.[68] The publisher tier, under which a capsule outside the project's namespace could run on its publisher's signature, admits no capsule in the release image.[69]

6.4 An example: the Linux personality

The capsule that matters most for this release is `capsule_linux`, the Linux personality. Its `Capsule.mk` requires the word `0x104001979`: `CoreExec`, `IPC`, `Memory`, `Crypto`, `FileSystem`, `Debug`, two graphics bits, `StoreWrite` and `ForeignExec`. It lists `Network`, `LocalSign` and `StreamImport` as optional, with the word `0x600000004`.[70] The kernel holds a copy of the same list, and a comment there says the manifest is what the gate enforces.[71]

Two things about that word are worth stating, because Section 7 depends on them. `ForeignExec` is the right to create a process the kernel has not verified, build its address space and answer its calls, and `capsule_linux` is the only capsule in the tree whose `Capsule.mk` requests it.[72] And `Network` is optional: it is held only by the instances that need it, which the personality's roles decide when they start, not by the program inside.[73]

[67] `src/kernel_core/process_spawn/capsule_spawn/runner/preflight.rs:36-76`
[68] `src/security/capsule_attest/against_root.rs:31-45`; `docs/security/stark-attestation.md`
[69] `docs/userland/signing-and-publisher-keys.md`
[70] `userland/capsule_linux/Capsule.mk:50-59`
[71] `src/userspace/capsule_linux/spawn.rs:36-55`
[72] `docs/security/capsule-isolation-the-linux-personalitys.md`
[73] `src/userspace/capsule_linux/roles.rs:32-58`

6.5 Other capsules relevant to keys

The keyring capsule holds the wallet's keys with no file, network or device access. It generates 12-word BIP-39 phrases, derives keys on `m/44'/60'/0'/0/i` for up to 8 accounts, and signs EIP-1559 transactions with in-process secp256k1, and it seals or opens the wallet's records only for a sender the service registry names as the wallet.[74] The one caveat that matters is old: any capsule that holds both `Crypto` and `FileSystem` can ask the kernel for the vault root under its published label and read the sealed blob under `/data`, and the Linux personality, Settings and the Terminal are among those that hold both.[75]

7 Running other people's programs

This section is the argument of the release. It has three parts: why a daily machine must run programs it cannot vouch for, how a capability microkernel turns a short signed list into enforcement, and what we saw when we tried to get past it.

7.1 Why the machine has to

The case for refusing everything we did not sign is easy to make and hard to live with. A machine that runs only the project's own capsules can be proved, in the sense of Section 5, down to the byte. It also has no password manager the person has used for years, no Telegram, no `git`, no Monero command-line wallet and no way to open a PDF that arrived by email. People route around a machine like that. They keep the real work on another computer, which is the computer the proofs were meant to protect them from.

So the machine has to run programs written by others, and the ones people want are not small. KeePassXC 2.7.8 is a Qt 5 application. Telegram Desktop 5.0.1 is Qt 6 with 254 packages in its install. Neither will be rewritten as a capsule. The honest options are to run them as they are, or not to run them.

Running them as they are raises a second question that most systems answer badly: what does the program get? On a typical desktop, a program started by the person has the person's authority. It can read their files, signal their other programs, open any socket and start any tool. The permission model, where there is one, is a set of dialogs answered by someone who cannot see what is behind them. A Linux password manager that wants the network and a Linux image viewer that wants the network are indistinguishable at the prompt.

NØNOS makes a different trade. A program gets a box, the box is a short list fixed in advance and signed with the program's Marketplace listing, and anything not on the list is refused with a message that says why. The person decides before install, by reading the list, not at runtime, by guessing.

7.2 Why a capability microkernel fits

Holding a program to a list is an old idea, and the usual trouble is where the check lives. In a monolithic kernel the check is a patchwork of hooks on the paths a program might take, and a missed path is a hole. A capability microkernel starts from the other side. A process has nothing unless it holds a capability, and almost every service a program could want is a message to a named capsule that has its own capability word.

That structure gives the box three properties, and they are the reason the design is built the way it is.

First, the guest holds nothing. A Linux program is a process created with an empty capability word. The kernel installs the word with `install_spawn(pid, 0)` and logs `[FOREIGN] guest pid=...`

[74] `userland/capsule_keyring/src/server/hd.rs:37-60; userland/capsule_keyring/src/server/vault_gate/mod.rs:17-25`

[75] `docs/security/protections-and-limits-what-nonos-does.md`

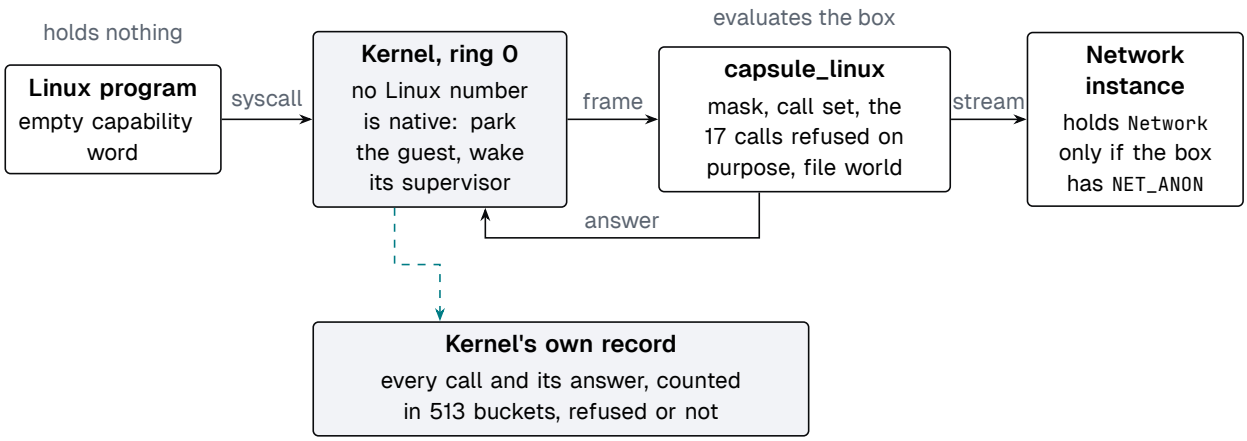


Figure 8. What happens to one Linux system call from a program in a box. The kernel parks the call and hands it to the one supervisor; the supervisor checks the call against the program's policy; the kernel counts the call and the answer in its own record. The program's native capability word is empty. Sources: `src/process/foreign/trap.rs`, `userland/capsule_linux/src/linux/app/`, `src/process/foreign/seal/`.

`caps=...` with the mask read back, so the log states what the guest holds instead of assuming it.[76] With no bits, every native call a guest could make fails the cap table. The one exception is `MkTtyQuery`, which only describes the caller's own streams.[77] A guest that learns NØNOS's system-call numbers gains nothing, because it has no token that admits them.

Second, every other call goes to one place. A Linux system call number is a small integer, and NØNOS native numbers are four ASCII letters, so the kernel does not recognise a Linux call. It parks the guest thread and wakes the capsule that created it, its supervisor.[78] The supervisor is `capsule_linux`, the personality. It is the only capsule that holds `ForeignExec`, and the kernel lets it touch only the guests it made: the peer calls that map, copy, protect and unmap guest memory begin with `supervised_asid`, which refuses any pid the caller does not supervise.[79] One supervisor holds at most 1,024 guests and threads.[80] So there is no path from a guest to the world that does not pass through the personality, and the kernel, not the personality, guarantees it.

Third, the checker is itself a capsule. It is signed, enrolled under the policy root and admitted by the STARK gate like any other, and it holds a fixed capability word. If the checker has a bug, the damage is bounded by that word, which Section 6 listed. This is weaker than it sounds and stronger than the alternative. It is weaker because the word includes `FileSystem` and `Crypto`. It is stronger because the checker is a small ring-3 program with a published word, not a patchwork inside the kernel.

Figure 8 draws the path of one call.

A word about what we do not say. It is tempting to write that the kernel checks the box on every call. The code says something more careful. The kernel guarantees that every call reaches the checker and that a guest has no other way out. The box itself, the mask and the declared call set, is evaluated by `capsule_linux`. The kernel also keeps a record of its own: it counts every call and its answer into 513 buckets, split into refused and not refused, as the answer is posted, so the record and the answer cannot part.[81] The two work together. The personality refuses a call outside the declared set with `EPERM`, which lands on the refused side of the kernel's count, so a record of a run can show that the

[76] `src/process/foreign/spawn_guest.rs:41-48`

[77] `src/syscall/contract/cap_table/mk_surface.rs:56-60`

[78] `src/process/foreign/trap.rs:28-57`

[79] `src/process/foreign/peer_guard.rs:64-73`; `docs/security/capsule-isolation-the-linux-personalitys.md`

[80] `src/process/foreign/room.rs:33-37`

[81] `src/process/foreign/seal/hist.rs:16-49`; `src/process/foreign/seal/absorb.rs:30-38`

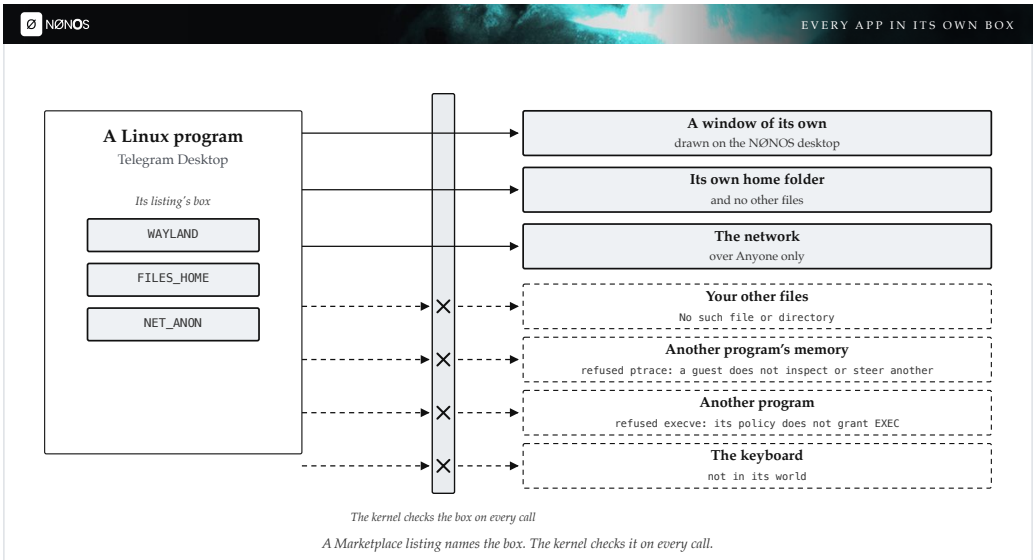


Figure 9. What a program in a box reaches, drawn for Telegram Desktop, whose listing grants `WAYLAND`, `FILES_HOME` and `NET_ANON`. Solid arrows are granted, dashed ones are refused, with the answer shown. The band's phrase, "the kernel checks the box on every call", is the project's; the text above and Figure 8 say how the checking is divided between the kernel and the personality. `NET_ANON` reaches the network the person chose, over Nym or Anyone, never Direct. Sources: `docs/using/linux-box.md`, `docs/using/marketplace.md`.

box held without trusting what the personality says about itself.[82] Whether that record is checked end to end on a booted release is a separate question, and Section 7.11 answers it.

7.3 What a box is

A box is 256 bytes. The layout is fixed: the magic `NLXPOL01`, a version, the BLAKE3 of the program, a mask of ten service bits, the listing and release ids, a hash of the destination list, and a bit for each of 512 Linux system-call numbers with one more for the native bucket. The decoder accepts exactly the bytes the encoder writes and refuses a wrong length, a non-zero reserved byte, a mask bit with no meaning, or a call bit past bucket 512, so a policy has one encoding and so one hash.[83] That hash is what a release signs. The **app id**, a BLAKE3 over the program's hash and the policy's hash, is what the personality notes in the kernel's record, so a proof names the program and its box together and neither can be swapped under the other.[84]

The ten mask bits are the whole vocabulary. Table 3 gives them with the number of the 31 catalogue programs that hold each.

`WAYLAND` A window of its own on the desktop, and the input routed to it. Without it, a program that opens the display socket gets `EACCES` and the personality logs `refused the display: its policy does not grant WAYLAND`.[85]

`FILES_HOME` Its own home folder. A guest's `/` is a tree in the file store, not the person's file system; `/home` and `/root` are private to the run and kept in memory; without the word they are the shared tree's, read and never written.[86]

`NET_ANON` Outbound streams, through the library that applies the person's chosen route, over Nym or Anyone and never Direct, even when Direct is the default network.[87]

[82] `userland/capsule_linux/src/linux/app/calls.rs:16-56`

[83] `userland/nonos_linux_policy/src/layout.rs:16-38`; `userland/nonos_linux_policy/README.md:1-20`

[84] `userland/nonos_linux_policy/src/identity.rs:16-40`

[85] `userland/capsule_linux/src/linux/unix/sock.rs:35-37`

[86] `userland/capsule_linux/src/linux/file/private/rule.rs:16-40`; `docs/userland/linux-personality.md`, section "The filesystem a guest sees".

[87] `userland/nonos_linux_policy/src/mask.rs:22-26`; `userland/capsule_linux/src/linux/net/guest_route.rs:59-75`

Table 3. The words a box is written in, and how many of the 31 Marketplace programs hold each. Three further words exist in the code: `FILES_USER` (the person's own files), `MODELS_RO` (reading the local models) and `SERVE_ONE` (one listener at a port a policy names); the catalogue generator refuses a recipe that asks for `FILES_USER`, so no listed program has it. Counted from `userland/capsule_market/linux-catalogue.json`; definitions in `userland/nonos_linux_policy/src/mask.rs`.

Box word	What it lets the program have	Programs
`WAYLAND`	a window of its own on the desktop	7
`TTY`	the Terminal	24
`FILES_HOME`	its own home folder, and no other files	31
`EXEC`	starting other programs, each held to the same box	8
`NET_ANON`	outbound streams, over Nym or Anyone only	8
`NET_LISTEN_LO`	listening on the machine's own loopback	3
`JIT`	making code as it runs	1

`EXEC` Starting other programs. Each program started is held to no more than the program that started it, so a chain of `execve` cannot widen the box.[88]

`TTY, NET_LISTEN_LO, JIT` The terminal; listening on loopback inside the program's own family; making code as it runs.

A program that arrives with no policy at all gets a default box: the terminal, its own home and `EXEC`, every call allowed, no window and no network.[89] The comment in the code calls this the owner's decision, and says that changing it is changing one constant. It is generous on purpose, so command-line tools keep working. It is also the box on the screenshot in Figure 12, because the programs run there were typed at the Terminal and have no listing.

7.4 From a listing to a running program

A listing is where the box is written down. A Marketplace listing for a Linux program carries the Alpine package, a pin of the package's BLAKE3, the program's licence, and a `guest_policy`: the 256 bytes above, written for the exact program the installer will record. The listing's release is signed by its publisher and the index by the Marketplace operator. A schema-3 Linux release with no policy answers **Missing** and can never be installed.[90]

An install goes through the six gates in Table 4, then fetches the package and what it needs one package at a time over the chosen route, checks each against the Alpine index the catalogue pins, and records the program whose bytes the policy names. A policy the market does not vouch for refuses the install. A failure names the package and the check that stopped it, for example `libzmq-4.3.5-r2 (its checksum did not match the signed index)`.[91]

The weakest word in that account is "proof". The attestation gate for a Linux listing says that the machine makes the proof itself after checking the bytes. That is the local-authority path of Section 6, a keyed tag, not a vendor STARK. What the person gets is a chain of checks on identity: the Alpine index signature, the catalogue's pin, the operator's and publisher's signatures, the bytes. The STARK stays with the project's own capsules.[92] We want the distinction to be plain, because it is the one a reader is most likely to blur. The proof is about NØNOS's capsules, and the box is about everyone else's programs.

[88] `userland/nonos_linux_policy/src/mask.rs:63-70`

[89] `userland/nonos_linux_policy/src/unlisted.rs:16-40`

[90] `docs/userland/marketplace-catalogue.md`; `userland/capsule_linux/src/linux/install/market/run.rs:34-62`

[91] `docs/using/marketplace.md`, section "How a listing is checked before it installs"; `docs/release/0.9.3-what-is-new.md`, section "Installing Linux apps".

[92] `docs/using/marketplace.md`, section "How a listing is checked before it installs".

Table 4. The six gates a Marketplace release must pass before an install is queued. The window decides nothing on its own: it queues a request, and the kernel asks the market for its verdict again. Source: `docs/using/marketplace.md`.

Gate	Passes when
index signature	the operator's signature over the catalogue verifies
package present	the release names a package URL, a package hash and a manifest hash
publisher signature	the release's publisher signature verifies
operator validation	the operator marked the release validated
architecture	the release runs on `x86_64-nonos` or `x86_64-linux` and needs kernel ABI 1 at most
attestation	the release carries a proof trailer hash, or it is a `linux.` listing, whose proof this machine makes after it has checked the bytes

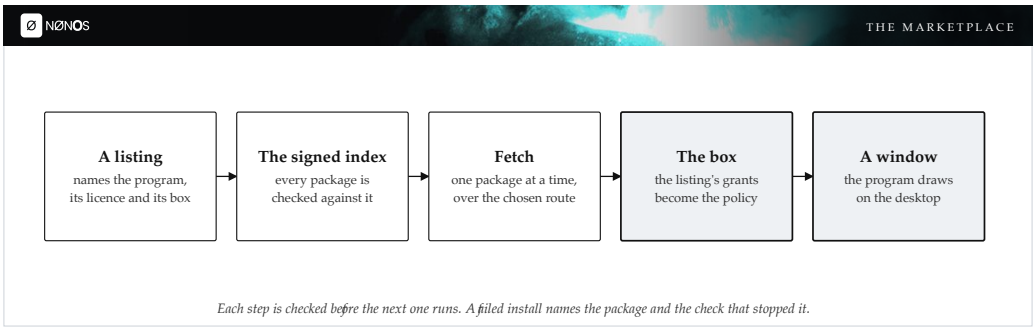


Figure 10. From a listing to a window. A listing names the program, its licence and its box; the package is checked against the signed index; the install fetches one package at a time over the chosen route; the listing's grants become the policy; the program draws a window. A failed install names the package and the check that stopped it. Sources: `docs/userland/marketplace-catalogue.md`, `docs/using/marketplace.md`.

7.5 Enforcement, one layer at a time

The box is enforced in three places, and each does a different job.

The kernel's part is the structure above: an empty word for the guest, one supervisor, peer access only to a supervisor's own guests, a count of every call. It does not know what a Linux call means. The record is explicit that it counts numbers only.

The personality's part is the box itself, and it is done call by call. Before a call is served, a bucket the policy leaves clear is `EPERM`, except `exit` and `exit_group`, which are served whatever the set says.[93] Beyond that, the personality holds a short list of calls it refuses on purpose with a stated reason: seventeen of them, among them `ptrace`, `process_vm_readv`, `process_vm_writev`, `mount`, `chroot`, `unshare`, `setns` and `io_uring`, which is refused as "a second call path around the gate".[94] A guest sees only its own processes in `/proc`, has no input devices, and cannot name the person's files because they are not in its world.[95] An internet socket needs one of `NET_ANON`, `NET_LISTEN_LO` or `SERVE_ONE`; `execve` needs `EXEC`.[96]

The network is enforced twice. The personality never holds `Network` in its ordinary run and terminal roles. A program whose box includes `NET_ANON` is started again in a different instance of the personality, `app.linux.run.net` or `app.linux.term.net`, which does hold the `Network` bit, and which

[93] `userland/capsule_linux/src/linux/app/rules.rs:21-31`; `userland/capsule_linux/src/linux/app/calls.rs:36-56`

[94] `userland/capsule_linux/src/linux/serve/refused.rs:21-47`

[95] `docs/release-evidence/0.9.3/s1-isolation-61ec29ea2/README.md`; `README.md`

[96] `userland/capsule_linux/src/linux/net/socket.rs:30-47`; `userland/capsule_linux/src/linux/call/spawn/exec.rs:28-31`

exits unless the policy it binds grants NET_ANON. A box without NET_ANON never runs there; the network role refuses it.[97][98] So for a program without the network word there are two independent refusals: the personality will not make an internet socket, and the instance serving it holds no capability with which the kernel would carry one. That second refusal is the kernel's, and it does not depend on the personality being right.

7.6 Windows

The new part of 0.9.3 that a person sees is that a Linux program opens its own window on the NØNOS desktop. The personality serves a Wayland display to a guest whose box grants WAYLAND. The guest speaks the Wayland wire protocol over a Unix socket the personality implements: registry, wl_shm buffers, xdg toplevels, seats and keymaps, data devices, output, decoration. The window is registered as a surface with the compositor through the two graphics bits the personality holds, and drawn inside the toolkit's own frame, with a NØNOS title bar and close, minimise and maximise buttons.[99] A surface's frame callbacks are answered at most 30 times a second, and at once for an idle window.[100]

The window is the cheapest thing in the box to give and the most delicate. A guest that holds WAYLAND can draw anything inside its own window and receive input routed to it. Whether one Linux window can reach another program's surface is a question we have not tested, and it is on the list of things we have not attacked.

What has been shown to run is narrower than the catalogue. On development images, swayimg drew its window, zoomed and closed; its frame's close button ended it with status 0; foot drew its window and ran a shell over /dev/pts; KeePassXC 2.7.8 drew its welcome screen in a box of WAYLAND and FILES_HOME, once the personality served FUTEX_WAKE_OP, which Qt 5 uses to wake a waiting thread. On a KVM run of the release tree, Telegram Desktop installed from the real Alpine mirror over Anyone and drew its welcome screen. No sign-in or sent message was shown. mpv and zathura are listed and no boot in the evidence shows a window for either. DarkFi's chat app is listed and no boot shows its window. Mousepad is not listed, because on the release tree it started in its box and showed no window.[101]

Dragging a Linux window by its title bar and minimising it are not shown. Copy and paste between a Linux window and the desktop is tested on the host only.[102]

7.7 Terminals

A terminal emulator opens a pseudo-terminal before it starts a shell, and until 0.9.3 there was none to open. A guest can now open /dev/ptmx, and /dev/pts/N opens the slave once it is unlocked. A family holds at most 64 pairs. What the master writes passes the slave's line discipline: input mapping, ^C as a signal for the foreground group under ISIG, line editing under ICANON, echo. A changed window size raises SIGWINCH. When the master goes, the slave reads end of file and its session leader gets SIGHUP.[103] A master writing with no reader stops at 4,096 pending bytes or lines and 64 KiB of output.[104] foot ran a shell over /dev/pts on a development image. That is the one booted use we can point to.[105]

[97] src/userspace/capsule_linux/run_net.rs:16-34; src/userspace/capsule_linux/role_term_net.rs:22-44

[98] userland/capsule_linux/src/linux/app/rules.rs:42-61

[99] userland/capsule_linux/src/linux/wayland/mod.rs:19-60; docs/release-evidence/0.9.3/gui-frame/README.md

[100] userland/capsule_linux/src/linux/wayland/pace_rule.rs:17-28

[101] docs/release/0.9.3-what-is-new.md, section "Linux programs in their own windows"; docs/release/0.9.3-known-issues.md, section "Linux windows".

[102] docs/release/0.9.3-known-issues.md, section "Linux windows".

[103] docs/userland/linux-personality.md, section "Pseudo-terminals"; userland/capsule_linux/src/linux/pty/table.rs:25-50.

[104] docs/release/0.9.3-what-is-new.md, section "Pseudo-terminals".

[105] docs/release-evidence/0.9.3/gui-foot/README.md

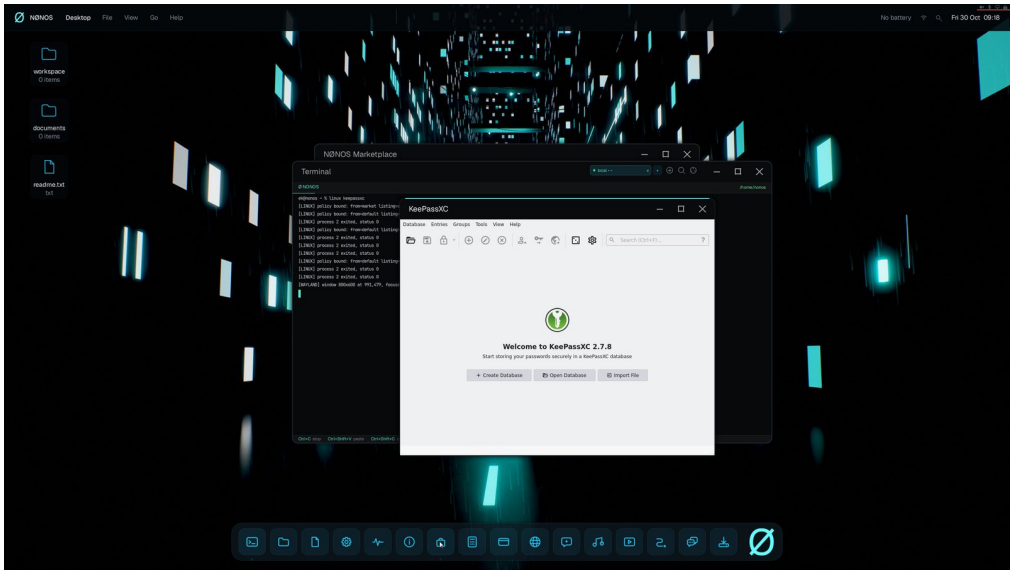


Figure 11. KeePassXC 2.7.8, a Linux password manager, drawing its own window on the NØNOS desktop, with the Terminal that started it behind it showing `[LINUX] policy bound` lines. Its box is `WAYLAND` and `FILES_HOME`: a window and a home folder, no network. Sealed 0.9.3 image under KVM. The screenshot shows a welcome screen; it does not show a database being made or opened.

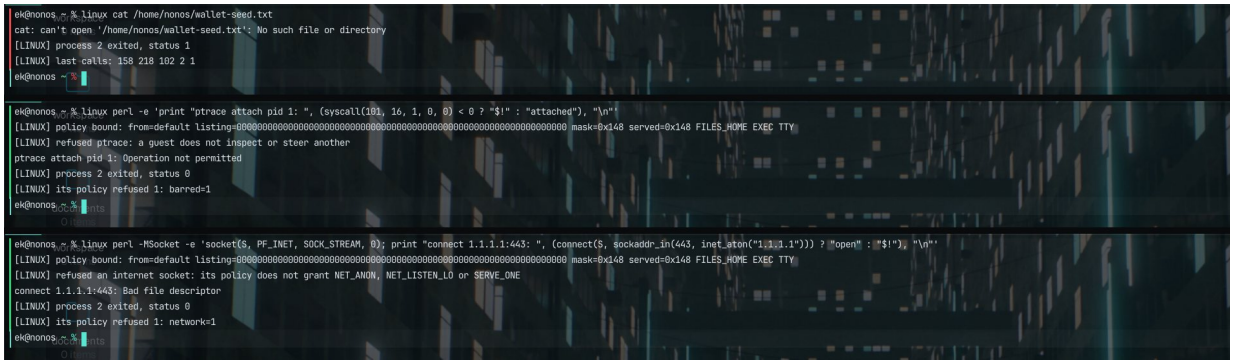


Figure 12. The Terminal after three attempts from inside a Linux program: reading a file that is not in its world, attaching with `ptrace` to process 1, and opening an internet socket and connecting out. The `policy bound` lines show `from=default`, a listing of all zeros and `mask=0x148`, which is `FILES_HOME`, `EXEC` and `TTY`: the default box of a program typed at the Terminal with no listing. Sealed 0.9.3 image under KVM, no TPM. Single attempts, as the next paragraph says.

7.8 The thirty-one programs

Table 5 lists the catalogue as it stands in the tree: 31 entries, each with its box and licence. One of them, the SimpleX Chat core, is an engine for a window that is not in this release and is not meant to be run directly, so 30 are things a person starts. The release notes count 30 recipes for that reason, and the README counts 31 rows.[106]

Two features of the table are worth reading before the table. First, every program holds `FILES_HOME`, and no program holds the person's files: the box for a text editor is a terminal and its own empty home. Second, only eight programs hold `NET_ANON`, and the ones that do are chat clients, wallets and Telegram. The shipped listings name no destinations, so a program with `NET_ANON` may open a stream to any host and port over the chosen route. A destination list is part of the policy format, and `linux --box` accepts one, but the catalogue uses it for none of its entries.[107]

7.9 What the kernel refused

We ran Linux programs on the sealed 0.9.3 image under KVM and tried the things a hostile program would try. Figure 12 is the Terminal after three of them, and Table 7 lists six, each with the answer the system gave.

These are single attempts from inside an app. They are not an exploit chain, and nobody has tried one. A program that is granted a right is allowed it: Telegram's box gives it the network, and it uses the network. We do not claim the box cannot be broken. What the attempts show is narrower and, to us, still worth having: the refusals are specific, they name the missing word, and they come from the code paths in Section 7 that we can point to.

Table 5. The 31 programs in the Marketplace catalogue with their boxes, grouped by what they draw. Generated from `userland/capsule_market/linux-catalogue.json` by `paper/tools/gen_tables.py`. Twenty-five come from Alpine v3.20 packages. The other six are not Alpine packages: DarkFi's four are built from source at a pinned commit, and SimpleX's two are the upstream release binary pinned by hash. A listing in this table is not evidence that the program has run: the status of each is in Section 12.

Program	Its box	Licence
With a window of their own (7)		
swayimg	`WAYLAND FILES_HOME`	MIT
Telegram Desktop	`WAYLAND FILES_HOME NET_ANON`	GPL-3.0-or-later + exc.
KeePassXC	`WAYLAND FILES_HOME`	GPL-2.0-only OR GPL-3.0-only
foot (terminal)	`WAYLAND TTY FILES_HOME EXEC`	MIT
mpv (media player)	`WAYLAND FILES_HOME`	GPL-2.0-or-later
zathura (PDF viewer)	`WAYLAND FILES_HOME`	Zlib
DarkFi (chat app)	`WAYLAND FILES_HOME NET_ANON`	AGPL-3.0-only
In the Terminal (23)		
Helix	`TTY FILES_HOME`	MPL-2.0
micro	`TTY FILES_HOME`	MIT
Kakoune	`TTY FILES_HOME EXEC`	Unlicense
Vim	`TTY FILES_HOME`	Vim
bat	`TTY FILES_HOME EXEC`	Apache-2.0
eza	`TTY FILES_HOME`	MIT
nnn	`TTY FILES_HOME EXEC`	BSD-2-Clause
ncdu	`TTY FILES_HOME`	MIT
fzf	`TTY FILES_HOME EXEC`	MIT
Git	`TTY FILES_HOME EXEC`	GPL-2.0-only
tig	`TTY FILES_HOME EXEC`	GPL-2.0-or-later
lazygit	`TTY FILES_HOME EXEC`	MIT
age	`TTY FILES_HOME`	BSD-3-Clause
minisign	`TTY FILES_HOME`	ISC
FLAC	`TTY FILES_HOME`	BSD-3-Clause AND GPL-2.0-or-later
LAME	`TTY FILES_HOME`	LGPL-2.0-or-later
Geth (dev chain)	`TTY FILES_HOME NET_LISTEN_LO`	GPL-3.0-or-later
Profanity (XMPP, OMEMO)	`TTY FILES_HOME NET_ANON`	GPL-3.0-or-later
Monero wallet (CLI)	`TTY FILES_HOME NET_ANON`	BSD-3-Clause
DarkFi wallet (drk)	`TTY FILES_HOME NET_ANON`	AGPL-3.0-only
DarkFi local dev chain	`TTY FILES_HOME NET_LISTEN_LO JIT`	AGPL-3.0-only
DarkIRC (P2P chat)	`TTY FILES_HOME NET_ANON NET_LISTEN_LO`	AGPL-3.0-only AND MIT
SimpleX Chat (terminal)	`TTY FILES_HOME NET_ANON`	AGPL-3.0-only
A background engine (1)		
SimpleX Chat core	`FILES_HOME NET_ANON`	AGPL-3.0-only

The kernel's side of the story is visible in the `audit` command, shown in Figure 13. It lists every process with the reach its capability word gives it (network, storage, devices, launch), who signed it and every bit it holds. Kernel threads show as not in the registry, with no signed ceiling recorded.[109]

[109] `userland/capsule_terminal/src/command/builtin/audit/table.rs:16-66`

Table 7. Six attempts from inside a Linux program on the sealed 0.9.3 image, and what the system answered. The first three are in Figure 12. The other three come from the project's README table of the same run, whose screenshot we do not have. The fourth, the refused `execve`, needs a box without `EXEC`, so it was not run in the default box of the figure.

Attempt	Answer
Read the person's files	`cat: can't open '/home/nonos/wallet-seed.txt': No such file or directory`; its `/home` is empty
Attach to another process	`[LINUX] refused ptrace: a guest does not inspect or steer another`
Open a connection out	`[LINUX] refused an internet socket: its policy does not grant NET_ANON, NET_LISTEN_LO or SERVE_ONE`
Start another program	`[LINUX] refused execve: its policy does not grant EXEC`
See other programs	`ls /proc` shows one process, its own
Read the keyboard	`cat /dev/input/event0`: no such device

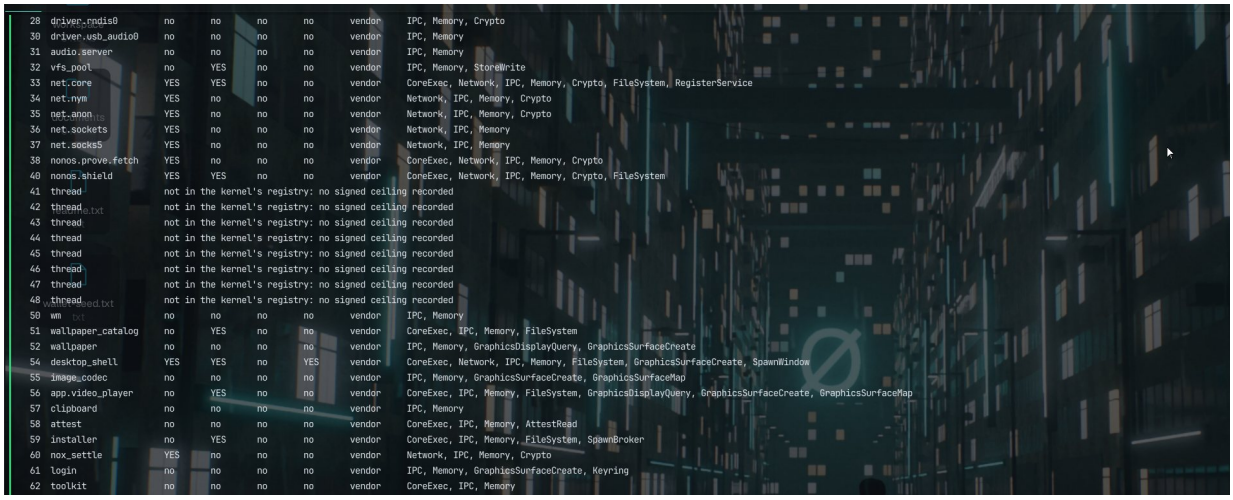


Figure 13. Part of the `audit` table on the sealed 0.9.3 image under KVM. Columns after the name are four reach areas (network, storage, devices, launch), the signer (`vendor` for every row shown) and the capability bits the process holds. Rows 41 to 48 are threads, which have no signed ceiling. The `devices` column names capability bits the kernel would honour, not devices claimed at the moment.

7.10 Running a program you wrote, or a model did

The same machinery is available by hand. `linux --box` runs a program in a named box and prints a report of what the box refused:

```
linux --box strict sh -c 'echo ok > /tmp/out; echo no > /home/out; nc 10.0.0.1 80'
linux --box offline python3 script.py
linux --box allow:/tmp/out,net:api.example.com:443 python3 agent.py
```

`strict` has no network and writes only to `/tmp`. An `allow:` box names its folders and, if it needs any, its destinations. A connection anywhere else is refused and counted, and the destination list is the one place in the current tree where a box can name hosts. `linux --box learn` runs a program once, enforced as `offline`, and writes the tightest box file that would have let it do what it did. The file is one run's behaviour, not what the program may need on another, and the documentation says to read it before use.[110] The report's counts are made by the personality as it refuses; they are not the kernel's histogram, and the page says so. This is the use we expect from people who run code a model wrote: a throwaway box, a printed list of what it tried, a file to tighten next time.

[110] `docs/using/linux-box.md`

7.11 Where the box is weak

The box is a design with a working implementation, not a finished wall, and we would rather say where it is thin than wait for someone else to.

The call set does not narrow anything yet. A recipe without a measured set is **unmeasured**: buckets 0 to 511 set and 512 clear, so every Linux call is allowed and only the native bucket is barred. None of the 31 catalogue entries carries a measured set. The machinery to enforce a narrower one exists and is tested on the host, and a listing could use it, but the boxes in Table 5 are, in practice, the mask plus the seventeen calls refused on purpose.[111]

The box is kept by path. The catalogue documentation lists three ways around it that are open today: a listed package's dependencies install unheld and run in the default box (tig pulls git, bat pulls less, vim pulls xxd); a held program copied to another path along with its trailer; and the loader invoked as `/lib/ld-musl-x86_64.so.1 <program>`, because the loader's gate checks only the trailer. Each ends in the default box, with every call allowed and no network. The proposed fix, tying a policy to the program's hash and not its path, is not in this release. We rely on that page at this commit; we did not re-run the three.[112]

The supervisor is a large target. It holds `FileSystem`, `Crypto`, `StoreWrite`, `ForeignExec` and `Debug` at all times, and the threat model says that authority follows the process, not the code. A program that takes over the personality gets those bits. One personality instance serves a run's guests together, a program and everything it starts. The personality's own documentation reports 369 host tests passing in the flake checks; the Linux surface is still a large amount of code to hold to a policy.[113]

A home does not survive power-off. A Linux program's home is held in memory, at most 16 MiB (64 MiB for the Monero wallet) and 128 names, and is gone at power-off. A home kept across a reboot is designed and not in the release, so the only lasting copy of a wallet seed is the one written down.[114] The design page says in its first line that kept homes move to the next release.[115]

The proof of a run has not completed on a boot. The Marketplace's proof of how a program ran timed out in two boots, and the 180-second wait that followed is not booted. Programs also fail in ways the tree states plainly: `lame` ended on signal 11 on a development boot, `micro` opened from the Marketplace stops on `open /dev/tty`, and `age`'s passphrase prompt, which writes to `/dev/tty`, does not show.[116]

Nobody outside the project has attacked it. The refusals above were produced by us, against our own code, on images we built. That is not an audit.

8 Cryptography

8.1 Signatures

Key files, certificates and manifests name the algorithm of each key in one byte: Ed25519, ML-DSA-44, ML-DSA-65 or ML-DSA-87. The kernel verifies only Ed25519 and ML-DSA-65, and the signer produces only those two.[117] ML-DSA-65, a FIPS 204 parameter set [1], comes from PQClean's portable C code under `third_party/pqclean`.[118] Requiring both algorithms means a forger must break both:

[111] `docs/userland/marketplace-catalogue.md`, section "The recipes file"; `userland/nonos_linux_policy/src/market.rs:31-43`.

[112] `docs/userland/marketplace-catalogue.md`, section "What a listing carries".

[113] `docs/security/protections-and-limits-what-nonos-does.md`; `docs/userland/linux-personality.md`, section "Tests".

[114] `userland/capsule_linux/src/linux/file/system/declared/home.rs:29-32`; `docs/release/0.9.3-known-issues.md`, section "Linux apps".

[115] `docs/design/kept-homes-and-installs.md`

[116] `docs/release/0.9.3-known-issues.md`, section "Linux apps".

[117] `src/crypto/asymmetric/alg_id/verify.rs:23-42`

[118] `build.rs:229-235`

Ed25519 against classical attackers and ML-DSA-65 against one with a quantum computer. The boot-root record and the kernel approval are signed with an ECDSA P-256 device-policy key. The kernel checks the record. The TPM checks the approval itself before it releases the device secret, and P-256 is a curve the TPM can verify. That key is not post-quantum, and we say so.[119]

Sizes: an Ed25519 public key is 32 bytes and its signature 64; an ML-DSA-65 public key is 1,952 bytes and its signature 3,309.[120] A manifest with both signatures therefore spends most of its bytes on the post-quantum one, and we think that is the right trade for a signature checked once at spawn.

8.2 Hashes and the STARK

Measurements use BLAKE3. The STARK layer uses Poseidon over the Goldilocks field for leaves and paths [2], and the proof system is transparent and hash-based [3, 4], so its security does not rest on discrete logarithms or pairings. The verifier is `nox_verify` from the STARKs repository, pinned by commit; no proof in the kernel tree covers its soundness.[121]

8.3 Keys the machine holds

The kernel signs capability tokens with a 32-byte key drawn at boot.[122] Machine keys are derived by the TPM under PCR policy and never stored (Section 5). Sealed records use ChaCha20-Poly1305 written in the tree, with keys from HKDF-SHA256 over the machine key and the record name.[123]

Three limits are documented. The TPM exchange that derives machine keys is unbound and unsalted, so on a discrete TPM the key crosses the bus in the clear. Labels are not tied to capsules, so any capsule holding `Crypto` can derive any machine-key label other than the kernel's own. And sealed records have no rollback protection: an older blob opens as well as a newer one.[124]

8.4 Entropy

0.9.2 reported that the generator serving `CryptoRandom` to capsules was seeded from RDRAND alone. The documentation now says otherwise, and we follow it. Both of the kernel's generators are seeded from the `entropy_pool` capsule's bytes mixed with RDSEED and RDRAND, and the capsule's bytes are never counted as enough alone: a seed needs 32 bytes from RDSEED and RDRAND, of which the capsule can supply at most 16. The loader adds a 32-byte seed of its own. The TPM is not a source.[125]

When no hardware source answers, the behaviour is the part to read. `get_bytes_secure` fails rather than fall back, but `fill_random_bytes` falls back to the cycle counter XORed with a counter, and a kernel with no hardware source still boots.[126] We have not found a check in the boot path that stops a machine with no source, and the documentation does not claim one.

9 Networking and privacy

9.1 Routes

The person chooses one of three routes in first-boot setup: the Nym mixnet, which is the default; the Anyone onion network; or Direct. One library, `nonos_route_link`, holds the rule for every program that

[119] `docs/security/boot-chain-and-signatures.md`, section "Keys and where their public halves live".

[120] `src/crypto/asymmetric/alg_id/lengths.rs:19-26`

[121] `Cargo.toml:1444`; `docs/security/stark-attestation.md`, section "What a trailer does not prove".

[122] `src/capabilities/token/material.rs:41-52`

[123] `userland/nonos_seal/README.md:1-12`

[124] `docs/security/protections-and-limits-what-nonos-does.md`; `userland/nonos_vault/README.md:32-34`

[125] `docs/security/randomness-and-cryptography.md`, section "Sources"; `docs/security/randomness-when-missing.md`.

[126] `src/crypto/random_api/basic.rs:42-46`; `docs/security/randomness-when-missing.md`

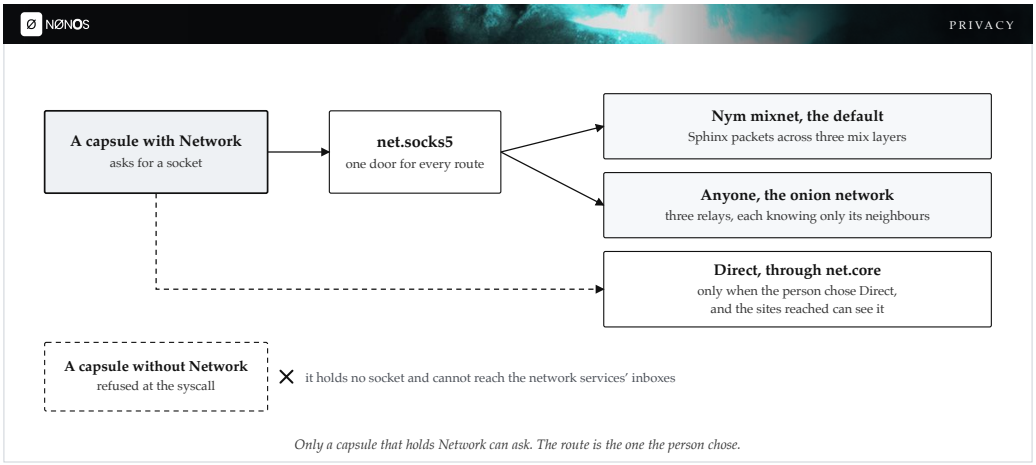


Figure 14. How a program's traffic leaves the machine. A capsule that holds `Network` asks `nonos_route_link` for a route; Nym streams go through `net.socks5` and `net.nym`, Anyone streams through `net.anon`, and Direct through `net.sockets`. A capsule without `Network` cannot reach the network services' inboxes. A Linux program in a box with `NET_ANON` takes the same library's route and is never given Direct. Sources: `docs/using/privacy-network.md`, `src/services/registry/policy.rs`.

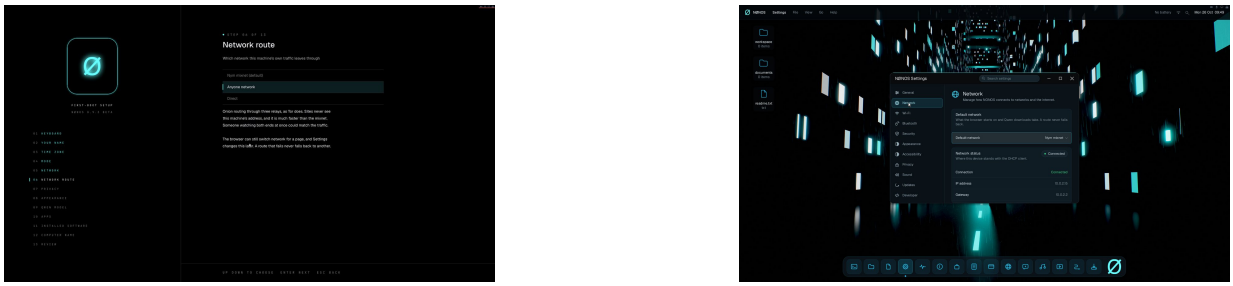


Figure 15. Left, the first-boot step that picks the network the machine's traffic leaves through. Right, Settings, which shows the network in use and changes it. Both from the sealed 0.9.3 image under KVM, so Settings shows a virtual network, not a real one.

opens its own connections. Direct is used only when Direct is chosen, Anyone goes through `net.anon`, and anything else, an unreadable setting included, goes through Nym. A chosen network that is not running means no route, and nothing falls back.[127] The wallet's chain reads and broadcasts never go Direct: under a Direct default they go over Nym, or over Anyone when Nym is not running, and with neither up there is no route.[128] Downloads for installs use Anyone whatever the default, because Nym's exits end long streams part way and Anyone's circuits carry gigabytes.[129] Host names travel unresolved to the exit, so no clear DNS query leaves the machine. Network drivers draw a new locally administered MAC address at each bring-up.[130]

9.2 What a Linux guest gets

For a Linux program the route logic is the same code with one extra rule. A stream to anywhere outside the program's own family goes by the system's chosen network: Anyone to `net.anon`, Nym or an unreadable setting to the mixnet socket, and Direct also to the mixnet, because a guest is never given a direct socket. A chosen network that is not running is `ENETUNREACH`, and nothing else is tried.[131] A program binds and listens only on 127.0.0.0/8; a datagram to outside the family is `ENETUNREACH`; a raw internet socket is `EPERM`.[132]

[127] `userland/nonos_route_link/src/pick.rs:130-139`; `docs/using/privacy-network.md`
[128] `userland/nonos_route_link/src/pick.rs:86-100`
[129] `userland/nonos_route_link/src/pick.rs:46-60`
[130] `docs/using/privacy-network.md`; `userland/nonos_mac/src/local.rs:46-55`
[131] `userland/capsule_linux/src/linux/net/guest_route.rs:59-75`
[132] `userland/capsule_linux/src/linux/net/policy.rs:33-53`

9.3 The transports

Both transports are new `no_std` code. NØNOS links neither the Nym client nor a Tor or Anyone client.[133] `net.nym` builds Sphinx [5] packets of 2,413 bytes for the Nym mixnet [6] with a 348-byte header across three mix layers and an exit gateway, and receives replies through single-use reply blocks. `net.anon` is a client for the Anyone network, a fork of Tor [7]: it checks consensus documents against directory-authority signatures, builds three-hop bandwidth-weighted circuits with the ntor handshake, and authenticates relays through Ed25519 certificates.[134] The browser, `curl`, `git` and the wallet use a `no_std` TLS 1.3 client with the Mozilla root set.[135]

9.4 What running traffic has shown

0.9.2 had to say that no booted image had carried traffic through a live Nym gateway or Anyone circuit. 0.9.3 has some, and we say what and where.

On a KVM host, the Monero command-line wallet synced from the node named in its listing, `xmr-node.cakewallet.com:18081`, over Anyone and over Nym. In one Nym run the serial log counts 3,321 `[SOCKS5] exit answered` lines while the wallet synced; in another run a fresh wallet refreshed to 3,794,280 of 3,794,280 blocks and `exit` ended it with status 0. These were development images, and the install itself used the host's package relay, so the install shows nothing about the real mirror; the sync does show the route. The wallet's own window progress is quoted in the evidence from the notes of those runs, because a terminal run's output never reaches serial.[136]

The same evidence is a good example of how these runs fail. An install over the real Alpine mirror stalled because the window opened three streams at once and both proxies keep one stream per caller, so each took the conversation from the one before and no package was answered. A lone stream carried the 24.7 MB Monero package whole. An install now fetches one package at a time. A failing first refresh on wallet open, "net.socks5 sent an answer this cannot read", is fixed on the host and has not been shown fixed on a boot, and the release notes tell the person to keep the seed on paper.[137]

A separate capture on 25 October judged the egress of an idle first boot of a development image under QEMU. It holds 2,287 frames, 2,284 of them from the guest under one random locally administered MAC. Every peer the guest reached is pinned in the image: five Nym bootstrap gateways, Nym's directory API and seven Anyone directory authorities. There was no DNS, NTP, discovery packet or IPv6. That is the nearest thing to a test of the setup wizard's sentence, "Telemetry: NONOS has none, so there is no switch for it", and it comes with its own list of what it did not show: nobody opened an app, no relay appears in the capture, and real hardware and Wi-Fi are not covered.[138]

9.5 What the network design does not cover

The kernel does not enforce the route in general. A capsule holding `Network` could call the TCP or socket services directly, and programs built on the toolchain's Rust `std` connect directly. Both transports start on every boot that starts networking, so the local network can see that a machine runs Nym and Anyone under every choice, including Direct.[139] Nym's directory is fetched over TLS from one validator without an operator signature, and cover traffic is sent only on request, which no program sets. `net.anon` has no vanguards, no conflux, no congestion control and no ntor v3, and does not check the digest in a relay's SENDME. Neither transport pads traffic on its own, and neither directory

[133] `docs/security/anonymity-transports.md`

[134] `docs/security/anonymity-transports.md`, section "Sphinx".

[135] `docs/security/tls-and-certificates.md`

[136] `docs/release-evidence/0.9.3/s1-g-kvm/README.md`; `docs/release/0.9.3-what-is-new.md`, section "The Monero wallet over Nym and Anyone".

[137] `docs/release-evidence/0.9.3/s1-g-kvm/README.md`; `docs/release/0.9.3-known-issues.md`, section "Linux apps".

[138] `docs/security/privacy-guarantees.md`, section "Booted evidence"; `userland/capsule_setup_wizard/src/render/screens/privacy.rs:18`.

[139] `docs/release/0.9.3-security-limits.md`

Table 8. Binding levels of the Lean modules, from `verification/lean/REFINEMENT.md`.

Level	Name	Meaning
1	extraction	the Rust is translated to Lean by Aeneas and the model is proved to refine it
2	differential	a proof crate includes the real Rust by `#[path]` and checks it against the model by tests and Kani
3	specification	the model is checked in Lean; there is no mechanical tie to the Rust yet

fetch is anonymous.[140] The documentation page for the transports still says that no booted image has carried live traffic, which the evidence above contradicts; we follow the evidence and have noted the page for correction.[141]

For Linux programs the position is better, because the route is enforced by the personality for guests and by the kernel for the instance that serves them (Section 7). For capsules in general it is a shared library and a convention that the capsules that hold `Network` follow.

10 Verification

NØNOS uses four kinds of evidence. Each means something different, and we keep them apart.

10.1 Lean 4 models, and how tightly they bind to the code

`verification/lean` holds 193 Lean files under Lean 4 [8] v4.15.0 without mathlib. A count of declarations introduced by `theorem` or `lemma` finds 1,519. Counting only plain top-level `theorem` lines gives 1,503, which is the figure recorded in `verification/evidence/EVIDENCE.json`; the README says 1,517. The three numbers come from three counting rules over the same files, and we use the widest. No `sorry` appears outside comments in any Lean tree, which we checked by search, and the core corpus declares no axiom beyond Lean's standard `propext`, `Classical.choice` and `Quot.sound`; the project's CI job fails on `sorryAx`.[142] We did not rebuild the proofs.

A separate tree, `verification/extraction`, holds about 2,200 further theorem declarations by the same widest rule, about functions that Charon and Aeneas [9] lowered from real Rust. EVIDENCE.json counts 978 extracted functions with 492 carrying a property of behaviour and 486 carrying only the theorem that a generated wrapper equals the method it forwards to. Eleven theorems there use `bv_decide`, which adds a generated axiom that asserts a SAT certificate checker, run as compiled code, answered correctly. The record says so, and says it never reaches the core corpus.[143]

What a theorem says about the running kernel depends on how the model is tied to the Rust. `verification/lean/REFINEMENT.md` defines three levels (Table 8) and states that a level 3 entry "is not a claim about the code". Only the capability modules sit at level 1.[144] Isolation, the capability table and dispatch, the IOMMU and the wipe at power-off are proved about models at level 3, with a proof crate or an extraction for some of their parts. The README's summary says Lean "proves no reachable state reaches off without" the wipe; that is a theorem about a model of the power-off path, and the reader should hear it that way.

[140] `docs/security/anonymity-transports.md`, section "What the transports lack".

[141] `docs/security/anonymity-transports.md`, section "What each transport reports".

[142] `verification/evidence/EVIDENCE.json`; `verification/lean/AxiomProfile.lean:1-20`

[143] `verification/evidence/EVIDENCE.json`

[144] `verification/lean/REFINEMENT.md`

10.2 Host proof crates, Kani and fuzzing

The tree has 223 directories ending in `_proofs` with a `Cargo.toml`, up from 56 at the 0.9.2 changelog. They mount shipped kernel or capsule source with `#[path]` and test it with `cargo test` on the host. EVIDENCE.json counts 222 as runnable; we have not found which of the 223 it leaves out.[145] There are 120 `#[kani::proof]` harnesses in the tree, 115 under `userland/` and five in the loader and attestation crates. Kani [10] runs in a separate workflow from the flake checks.[146] Test counts the documentation quotes, for example 388 for `kernel_proofs`, 198 for the in-tree `stark_proofs` and 369 for the Linux personality's pure modules, are the documentation's, from the flake checks recorded at its own commits; we did not rerun them.[147]

10.3 Static checks and adversarial tests

Python checkers hold the published ABI to the kernel: one compares the gate of each system call, one the capability bits, one each capsule's mask against the calls its code makes.[148] An attack capsule, `userland/capsule_attack`, tries every row of the isolation matrix with raw system calls; no image carries it. A selftest image, which boots a capsule that tries what an attacker would, reported `attack suite: PASS` on a development image under QEMU, including a refused attempt to use the shell's open-with path as a deputy.[149]

10.4 Reproducible builds and receipts

The build is pinned by hash and runs offline in Nix's sandbox. `make` ends with a receipt of every artifact by SHA-256 and prints `REPRODUCED` when the bytes match the committed receipt.[150] Sealed images differ by design, because signatures and trailers differ between seals. A comparison of builds on two machines is not tested in this release, and the documentation lists what is not reproducible yet.[151]

10.5 What has not run

Several checks that matter for the trust path did not run for this release. No flake check runs `nox_verify` over a real proof; the enrolment tool's selftest does, and no run of it is recorded for this commit. The booted refusal test, which expects four STARK refusals and one admission under QEMU, was not run. The Kani harnesses for the attestation path crate were not run. The Marketplace's proof of how a program ran has not finished on a boot. And the claims in Section 7 about the box rest on boots of development images and one sealed image, not on a proof.[152]

The public verifier at `verify.nonos.software` runs `nox_verify` compiled to WebAssembly over the 0.9.2 release's enrolled units. We did not check what it lists for 0.9.3, and this paper does not rely on it.

11 The Ethereum angle

This section separates what exists from what is a direction. Nothing in it describes a shipped product for machine attestation on Ethereum. It is shorter than in 0.9.2, because the kernel repository holds

[145] `verification/evidence/EVIDENCE.json`

[146] `docs/contributing/tests-and-proofs.md`

[147] `docs/security/capsule-isolation-the-linux-personalitys.md`; `docs/security/stark-attestation.md`, section "Tests".

[148] `scripts/check_syscall_caps.py`

[149] `docs/overview/threat-model-assets.md`; `docs/release-evidence/0.9.3/ksec-d2a9d0bfa-iommu-intel/README.md`

[150] `receipts/README.md`

[151] `docs/build/reproducible-builds.md`

[152] `docs/security/stark-attestation.md`, section "Tests"; `docs/release/0.9.3-known-issues.md`.

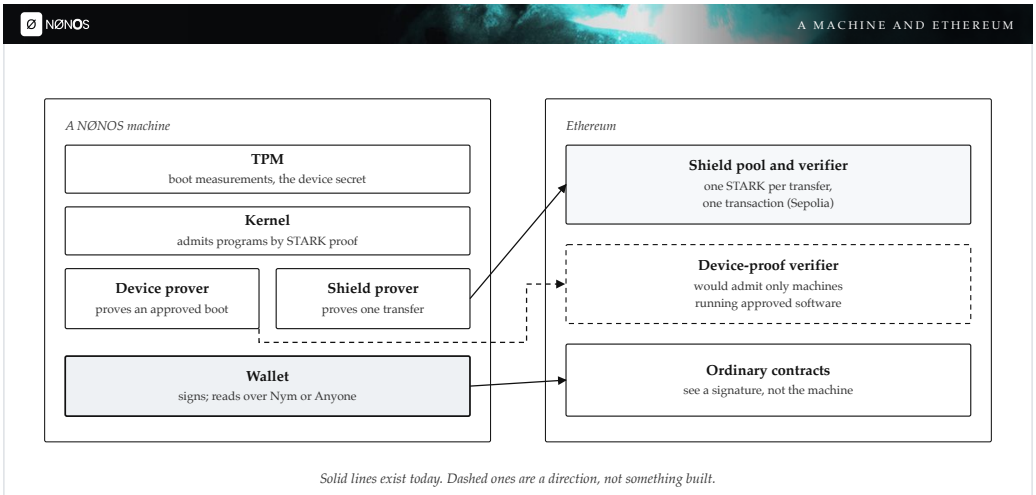


Figure 16. What connects a NØNOS machine to Ethereum, and what does not exist yet. Solid lines exist today; dashed ones are a direction, not something built. Sources: `docs/security/stark-attestation.md`, `nonos-device-attest/README.md`, `abi/NOX_CONTRACTS.md`.

no Solidity and nothing in this release moved the direction forward. We keep it because the question is why the project started, and because the box of Section 7 sharpens it.

11.1 The weak link

Ethereum secures its state with consensus and with code that every node executes. The machines at its edges are outside that guarantee. A wallet machine that runs a compromised program can sign whatever the program asks. A validator's signing key is only as safe as the server that holds it. An RPC provider sees which accounts a given network address asks about, and that links a person's location to their holdings. A contract can check a signature. It cannot learn anything about the machine that produced it.

The box adds a smaller point. A wallet that reads a seed from a file is only as private as every program that can read that file. In NØNOS a program in a box has no access to the person's files at all. That is a property of the endpoint, and no on-chain rule can see it.

11.2 What exists today

A NØNOS machine can produce proofs about itself. Three artefacts exist, all checked off-chain. The admission proofs: every enrolled kernel, loader and capsule carries a STARK of membership under a public root. The TPM quote from `MkAttestDoc`, which names the machine and is withheld from any capsule with network rights. And the anonymous device proof of Section 5, whose own README lists it as built and never run on a booted image, and which no verifier service or contract consumes.[153]

The wallet hides the RPC link. The wallet's chain reads go over Nym or Anyone, with host names resolved at the exit, and never over Direct (Section 9).

The NOX token has a contract set specified in the tree. `abi/NOX_CONTRACTS.md` describes a contributor-reward system: off-chain receipts for work such as boots and capsule builds, aggregated into Merkle roots and paid from reward pools, with a deployment record on Ethereum mainnet under a 3-of-5 Safe. We did not query the chain to confirm the deployment, and the Solidity is not in this repository. The page is explicit that "v1 verification is off-chain with open tooling" and that v1 does no on-chain ZK verification. In 0.9.2 the same page listed ten contracts; it now lists eight, and we report it as the document stands.[154]

[153] `docs/security/stark-attestation.md`; `nonos-device-attest/README.md`

[154] `abi/NOX_CONTRACTS.md`; `abi/nox_deployment.json`

The Shield pool, which settles a private transfer with one STARK checked by a verifier contract, is described in a separate paper [11] and in repositories that are not part of this tree. We cite it and make no claim about it that we checked here. Its authors state that it has not been audited.

11.3 What does not exist

No contract verifies anything about a machine. No deployed contract checks an admission proof, a TPM quote or a device proof, and no transaction signature produced on NØNOS is bound to an attested state. The project's public plan lists attested clients as an on-chain fact under work it is not building.[155]

11.4 The direction

The pieces suggest a path to a machine that proves to a contract what it runs. We state the steps and what each would need, without claiming any.

1. A verifier contract for the device proof. It proves a different statement over a different trace from the Shield pool's, so it would be a new contract with its own gas cost and its own audit.
2. Roots on chain: the release's bootloader, kernel and device-registry roots, from a source a contract trusts, such as a registry updated by the release's multisig.
3. Scope and freshness. The statement already binds a verifier scope and a nonce and yields one tag per device per scope, so a contract could use its own address as the scope and a recent block hash as the nonce.
4. Binding an action to the proof. The statement has no field for a public key or a message, so letting a contract accept a signature only from an attested machine would change the circuit.

Two limits would remain. The proof shows that an approved chain exists and that the prover holds an enrolled device secret; that this machine booted that chain rests on the TPM releasing the secret only in that boot state, and so on the TPM and firmware being honest. And the proof says nothing about the behaviour of the code it admits, only its identity. Section 7 is the other half of that sentence: identity of the system's own code from the proof, behaviour of everyone else's from the box. A contract that wanted to know that the signing machine could not be reached by a stray program would need both.

12 Status

Table 9 gives the state of each component at the studied commit, from evidence in the tree. We use four columns. **Built**: the code is in the tree and in the release artifacts. **Booted**: a boot exercised it, and the cell says which kind. **Hardware**: a run on physical hardware is recorded. **Audited**: an external party reviewed it. No component has been externally audited. The release notes say that this release claims no third-party security audit, and the bug-bounty policy is a draft.[156]

Two kinds of boot matter and are easy to confuse. A **development image** is built with throwaway keys and path-only attestation, under QEMU. Most of the more than fifty evidence directories are of that kind. A **sealed image** is the release build, signed and enrolled with the release keys. The screenshots in this paper are from a sealed image under KVM, which gives the guest no TPM, so the TPM paths did not run in them.[157]

12.1 What did not run

The documents we rely on list what did not run, and we repeat the parts that bear on this paper. The booted STARK refusal test was not run. Booting with Secure Boot on is not tested, and neither is an

[155] `https://nonos.software/roadmap/`

[156] `docs/release/0.9.3-security-limits.md`; `docs/overview/mission.md`

[157] `docs/release-evidence/0.9.3/README.md`

Table 9. Component status at the studied commit. "Dev QEMU" is a development image under QEMU; "sealed KVM" is the sealed 0.9.3 image under KVM, with no TPM; "swtpm" is QEMU with a software TPM. "Host" means host test suites only. Real hardware means the owner's and the maintainer's reports in the hardware matrix, one Intel Gemini Lake laptop for the maintainer's. No row has been audited.

Component	Built	Booted	Real hardware
Loader: signatures, kernel STARK, rollback	yes	sealed KVM (no TPM, so no rollback floor)	not recorded
Kernel check of its loader (TCG log)	yes	not recorded; needs a TPM	not recorded
Capsule spawn gate, STARK half	yes	owner's sealed boot under QEMU: every capsule attested as `vendor`; refusal test not run	not recorded
Capability check, 183 calls	yes	every boot; host tests	implied
IPC gates and budgets	yes	host	implied
IOMMU confinement, Intel VT-d	yes	dev QEMU with a virtual IOMMU: enforcing, no unconfined grant	not recorded
IOMMU, AMD-Vi	yes	QEMU's AMD IOMMU only	no
TPM keys, handover, update in place	yes	swtpm: a prepared volume opened across a changed bootloader	not recorded
Secure Boot, Hardened profile	yes	not tested	not tested
Linux box: refusals	yes	sealed KVM and dev QEMU; single attempts	no
Linux windows	yes	swayimg, foot, KeePassXC, Telegram drew windows; mpv, zathura, DarkFi app not booted	no
Pseudo-terminals	yes	foot's shell, dev QEMU	no
Install from the real Alpine mirror	yes	KVM: 254 Telegram packages over Anyone with 4 and 8 GiB	no
Monero wallet over Nym and Anyone	yes	KVM, dev images: synced over each	no
Nym and Anyone transports	yes	above, and a capture of an idle boot's egress	no
Wired Ethernet, 7 models	yes	DHCP lease on dev QEMU; no traffic past it shown	no
Wi-Fi RTL8821CE	yes	no	owner's report, no log
Other Wi-Fi drivers	yes	modelled devices only	no
Power-off wipe	yes	dev boot: a test secret in RAM 8 times before, none after	no
Anonymous device proof	yes	host only; never booted	no
Lean models	1,519 theorems	n/a	n/a

update from one release to another. A disk made before 0.9.3 cannot be kept, and its installer says so. The proof of how a Linux program ran timed out in two boots. mpv, zathura and DarkFi's window have not been shown. Telegram's sign-in has not been shown. A real-mirror install over both routes, one package at a time, was still being run when the evidence was written. The anonymous device proof has never run on a booted machine.[158]

The owner's first sealed image of 26 October was judged no-go after a kernel heap exhaustion following lock waits in the paging manager. The lock fix is booted in eight boots at 4 and 8 CPUs with no lock waits. We report this because a status section that omits its failed release candidate is not one.[159]

[158] `docs/release/0.9.3-known-issues.md`; `docs/release/0.9.3-upgrade-notes.md`

[159] `docs/release-evidence/0.9.3/owner-sealed-crash/README.md`; `docs/release-evidence/0.9.3/s1-paging-lock-c3f522575/README.md`

13 Use cases

The project's site lists concept pages in ten sectors, each marked as a deployment concept that needs hardware support, integration and validation.[160] We pick four where the mechanisms in this paper apply directly. For each we name what it replaces and what is missing. None should carry real value before the system has been audited.

A daily machine that is not a trap. What it replaces: a laptop where every program the person installs has the person's authority. On NØNOS a password manager, a chat client and a PDF viewer each get a box that is visible before install: KeePassXC has a window and a home and no network; Telegram has a window, a home and the chosen anonymity network; neither can see the other's memory or the person's files. What is missing: a home that survives a reboot, so for now every such program starts empty, and Telegram's sign-in is not shown to work.

Running code a model wrote. What it replaces: pasting a script into a shell. `linux --box strict` runs it with no network and a scratch folder and prints what the box refused; `learn` writes the tightest box file for what one run did. What is missing: a destination list is enforced in `linux --box` but no listing carries one, and the weaknesses of Section 7.11 apply.

A signing machine for keys that hold value. What it replaces: a general-purpose laptop, or a hardware wallet paired with one. The key lives in a keyring capsule with no file, network or device rights; only the wallet window may ask it to sign; the wallet's chain reads never leave in the clear; and every capsule on the machine was admitted by a STARK proof and runs with a word fixed at enrolment. The air-gapped profile removes every network driver and service from the image. What is missing is a trusted display path from the transaction decoder to the screen.

An attested endpoint for validators, oracles and agents. What it replaces: trust in an operator's statement of what runs on a server. A NØNOS machine can produce a TPM quote and a device proof, and Section 11 sets out how far this is from a contract that checks one. The project lists it as a direction it is not building.

14 Limitations

Section 12 lists the evidence gaps and Section 7.11 the weaknesses of the box. The design and implementation limits below are the ones the release notes and security pages document for 0.9.3.[161]

- Firmware, CPU and TPM are trusted. With Secure Boot off, a whole image sealed with other keys passes the loader's and the kernel's checks, and the TPM then gives it a different machine key. Booting with Secure Boot on is not tested.
- Without a TPM, rollback protection is off in every mode except Hardened and Air-Gapped, and the kernel's check of its loader rests on the loader's own word.
- On a machine whose firmware left VT-d or AMD-Vi off, a Standard boot goes on with device DMA unconfined and says so. On a Standard boot, two vendor drivers may share an isolation group when only a port without ACS joins their devices. AMD-Vi has run only under QEMU. Physical memory above 64 GiB is not used.
- The image-wide capability ceiling is computed and logged, and a capsule that exceeds it still starts. `IO` and `Hardware` gate nothing. Any capsule with `Crypto` can derive machine keys under other capsules' labels. Authority follows the process, not the code: an attacker who takes over a capsule gets every capability it holds.
- The wipe at power-off misses kernel statics: the data volume key, the token MAC key and the kernel's in-memory copy of its console lines. It also misses a live stick's in-memory volume. A kernel panic wipes the kernel heap and powers off, but not process memory. A forced power-off or a power cut wipes nothing.

[160] `https://nonos.software/use-cases/`

[161] `docs/release/0.9.3-security-limits.md`; `docs/security/protections-and-limits-what-nonos-does.md`

- The TPM exchange that derives machine keys is unbound and unsalted, so a key crosses a discrete TPM's bus in the clear.
- From the moment the installed system prepares an update until the next boot, a kernel the release approved opens the data volume on this machine's firmware and Secure Boot state, whatever bootloader measured it. With Secure Boot off, a bootloader that measures an approved kernel and then starts another would open it, and a copy of the disk taken in that window keeps the handover record.
- The package store is written to the disk unencrypted. Only the data volume, and records a program seals itself, are protected there.
- The kernel image sits at a fixed address, the kernel has no stack canaries, and the kernel stack a process makes its system calls on has no guard page. KPTI is not implemented and the L1D flush is never issued, so Meltdown-affected parts are not mitigated. Side channels between capsules are out of scope.
- `net.nym` and `net.anon` start on every boot that starts the network, so the local network can see that the machine runs Nym and Anyone. The kernel does not enforce the route for capsules in general.
- A Linux program's home is its own and lasts until power-off. Nothing an app keeps survives a reboot.
- The STARK's soundness is a claim of the STARKs repository, outside the kernel tree's proofs, and Lean's isolation results are about models.
- Counts in the project's own documents disagree with the code in places: the syscalls page says 176 where the ABI file says 183, and the architecture page counts 107 capsule directories where the tree has 152. We follow the code.
- The project lists its team as two people.[162]

What none of these lists contains is an outside attacker's findings. The first external review will find things that this paper does not.

15 Where the work goes next

Everything in this section is design or intent. Each item names the document in the tree that states it and the status that document gives itself.

Make the box follow the program, not the path. The three known ways around the box (Section 7.11) all come from keeping a policy by path. The catalogue page says the fix is to tie a policy to the program's hash. This is the first thing we would do, because a box that can be walked around by copying a file is a weaker claim than the rest of this paper makes.[163]

Measure the call sets. The machinery to hold a program to the calls it was seen to make exists, and a recipe can carry a measured set. None of the 31 does. Measuring them means running each program through its real uses, and the number of calls a Qt application makes is large, so this is a slog and not a research problem.[164]

Homes that survive a reboot. Status of the design page: the owner ruled on 27 October; 0.9.3 ships the handover and the in-place update, and kept homes and installs move to the next release. The design is that a home is sealed under the data volume's key and opens only on this machine. Until it lands, a wallet seed is safe only on paper.[165]

A secret service inside each box. Proposed, before code, for the next release: a place for a program's saved passwords that is not its home folder.[166]

[162] `https://nonos.software/roadmap/`

[163] `docs/userland/marketplace-catalogue.md`, section "What a listing carries".

[164] `docs/userland/marketplace-catalogue.md`, section "The recipes file".

[165] `docs/design/kept-homes-and-installs.md`

[166] `docs/design/box-secret-service.md`

One listener, and OpenGL on the CPU. `SERVE_ONE`, one guest port for one capsule, is approved and in code, and `box-gl` puts Mesa's software renderer behind a program that asks for OpenGL. Both are marked "0.9.3 if it is done properly, else 0.9.4"; the `SERVE_ONE` bit is in the policy format already, and Mesa in the release's Linux tree is not.[167]

A quote that names the NØNOS chain. The attestation document quotes firmware registers and not the ones NØNOS extends. Version 5 of the document is a design that quotes the loader, the kernel and the proof measurements, with nothing landing before the 0.9.3 seal.[168]

A started capsule never holds more than its starter. A design for after the seal, with a minimal fix proposed for 0.9.3 itself and no kernel code yet.[169]

Outside review. The most useful item is not in the tree. A person who did not write the box, with a week and a hostile program, will find more than we did.

16 Related work

seL4. seL4 is a capability microkernel with a machine-checked proof that its C implementation refines an abstract specification, and proofs of integrity and confidentiality on top [12, 13]. NØNOS shares the capability model and the ring-3 drivers. Its proofs are much weaker in binding: only the capability modules reach extraction, and isolation is proved about models. It adds things seL4 leaves to the system built on it: an admission gate with STARK membership proofs and post-quantum signatures, measured boot checked in both directions, and a desktop and network stack.

Fuchsia, Zircon and Starnix. Zircon is a capability-based kernel in which processes hold handles with rights and drivers run in user space [14]. Fuchsia's Starnix runs Linux programs on Zircon by implementing Linux system calls in a user-space component [15]. That is the closest relative of the Linux personality, and the shape is the same: the kernel does not speak Linux, a supervisor does. The difference we can point to is the unit of policy. We tie a short signed word to each program's listing and hold the personality's authority to a published capability word. We do not claim that this is better than Starnix's arrangement. We have not compared them.

gVisor. gVisor is an application kernel that intercepts a container's system calls and serves them in user space, to narrow what reaches the host kernel [16]. The personality is the same idea on a smaller host: there is no Linux kernel underneath to protect, because NØNOS is not Linux. gVisor has far more of Linux behind it than we do. The personality serves 226 of 373 x86_64 calls by the tree's own coverage tool, and refuses seventeen on purpose.[170]

Capsicum. Capsicum adds a capability mode to FreeBSD, in which a process loses access to global namespaces and holds rights on file descriptors [17]. The idea that a program gets a short list of rights and nothing ambient is the same. Capsicum asks the program to be written for it, or wrapped. A NØNOS box is applied to a program that was written for a world of ambient authority, by giving it a world that is mostly empty.

Flatpak. Flatpak sandboxes desktop applications using Linux namespaces and seccomp, with the permissions an application asks for declared in its metadata, and portals for the rest [18]. The declared list and the Marketplace listing play the same role. The difference is the layer: Flatpak's sandbox is a set of kernel features inside a large kernel, and ours is a small kernel plus one supervisor. Flatpak is mature and widely used; the NØNOS box is new in this release.

Qubes OS. Qubes isolates workloads in Xen virtual machines and routes networking through service VMs, including Whonix for Tor [19]. Its isolation boundary is a hypervisor running full guest kernels. NØNOS runs one small kernel and confines each program by capability, with no guest kernels. The cost is that the compatibility layer is ours to write, and Qubes inherits Linux's.

[167] `docs/design/serve-one.md`; `docs/design/box-gl.md`

[168] `docs/design/k1-attest-doc-v4.md`

[169] `docs/design/k2-attenuated-spawn.md`

[170] `tools/nonos-linux-coverage` prints 226 of 373; `docs/userland/linux-personality.md` still says 224.

Tails. Tails is an amnesic live system that routes all traffic through Tor and keeps nothing by default [20]. NØNOS shares the amnesic default and the refusal to fall back to the clear network, adds Nym as a default transport, and moves enforcement from a firewall to the route library and, for Linux programs, to the personality.

GrapheneOS. GrapheneOS hardens Android with verified boot, hardware-backed key attestation and a sandboxed app model [21]. Its attestation relies on the phone's secure element and the vendor's certificate chain. NØNOS's admission proof is checked against a public policy root, and anyone can check it.

TPMs and trusted execution. The TCG specifications define PCR extension and quotes [22, 23]. A quote proves what was measured to a verifier who knows the expected values. NØNOS adds a membership proof so that the expected set is fixed by one root, and an anonymous device proof that hides which member is running. Trusted execution environments such as SGX, TDX and Nitro Enclaves let a remote party verify a measurement signed by hardware keys rooted in the vendor [24, 25, 26]. They protect a workload from its host but depend on vendor key infrastructure. NØNOS takes the opposite position: it trusts the CPU and TPM it runs on and makes the software side open and checkable by proofs that need no vendor key.

References

[1] NIST. FIPS 204, module-lattice-based digital signature standard. https://csrc.nist.gov/pubs/fips/204/final, 2024.

[2] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: a new hash function for zero-knowledge proof systems. In **USENIX Security**, 2021.

[3] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046, 2018.

[4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In **ICALP**, 2018.

[5] George Danezis and Ian Goldberg. Sphinx: a compact and provably secure mix format. In **IEEE Symposium on Security and Privacy**, 2009.

[6] Claudia Diaz, Harry Halpin, and Aggelos Kiayias. The Nym network. Whitepaper, https://nymtech.net/nym-whitepaper.pdf, 2021.

[7] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: the second-generation onion router. In **USENIX Security**, 2004.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In **CADE**, 2021.

[9] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. In **ICFP**, 2022.

[10] Kani Rust verifier. https://model-checking.github.io/kani/.

[11] NØNOS. NØNOS shield: private transfers on Ethereum L1, proved on the device and verified by a direct STARK, v1.0. https://nonos.software/assets/papers/shield-whitepaper/v1.0/nonos-shield-whitepaper.pdf, 2026.

[12] Gerwin Klein et al. seL4: formal verification of an OS kernel. In **SOSP**, 2009.

[13] Toby Murray et al. seL4: from general purpose to a proof of information flow enforcement. In **IEEE Symposium on Security and Privacy**, 2013.

[14] Google. Fuchsia: Zircon kernel concepts. https://fuchsia.dev/fuchsia-src/concepts/kernel.

[15] Google. Fuchsia: Starnix. https://fuchsia.dev/fuchsia-src/concepts/starnix.

[16] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The true cost of containing: a gVisor case study. In **HotCloud**, 2019.

[17] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: practical capabilities for UNIX. In **USENIX Security**, 2010.

[18] Flatpak documentation: sandbox permissions. https://docs.flatpak.org/en/latest/sandbox-permissions.html.

[19] Joanna Rutkowska and Rafal Wojtczuk. Qubes OS architecture. Invisible Things Lab, 2010, https://www.qubes-os.org.

[20] Tails: the amnesic incognito live system. https://tails.net.

[21] GrapheneOS: features overview. https://grapheneos.org/features.

[22] Trusted Computing Group. TCG PC client platform firmware profile specification. https://trustedcomputinggroup.org.

[23] Trusted Computing Group. TPM 2.0 library specification. https://trustedcomputinggroup.org.

[24] Victor Costan and Srinivas Devadas. Intel SGX explained. IACR ePrint 2016/086, 2016.

[25] Intel. Intel trust domain extensions (TDX) overview. https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html.

[26] Amazon Web Services. AWS Nitro enclaves user guide. https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave.html.

[27] NØNOS microkernel-preview repository, branch main. https://github.com/NON-OS/microkernel-preview. Commit 832ab24778, 30 October 2026.

[28] NØNOS website: home, documentation, use cases, roadmap. https://nonos.software.